

VHDL: a Hardware Description Language for Simulation and Logic Synthesis

1. Introduction: Why HDLs?

2. Design Flow and Tools

3. Basic Language Concepts

4. Signal and Delay Models

5. Modeling Digital Systems

6. Concurrent and Sequential Processes

Process statement, process event behavior, signals vs. variables, timing behavior of processes

7. Modeling Structures

Structural models, generics, the Generate statement

8. Simulation and Validation

Concepts, writing testbenches, configurations

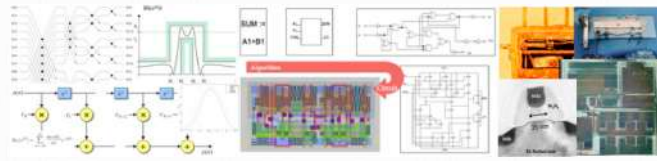
9. RTL and Logic Synthesis

Writing style for logic synthesis, combinational logic, sequential logic, RTL and logic synthesis CAD algorithms

10. Fil Rouge Example: FIR filter

Olivier Sentieys
ENSSAT - Université de Rennes 1
IRISA/INRIA
sentieys@irisa.fr

http://people.rennes.inria.fr/Olivier.Sentieys/?page_id=95



VHDL: a Hardware Description Language for Simulation and Logic Synthesis

Olivier Sentieys

ENSSAT - Université de Rennes 1

IRISA/INRIA

sentieys@irisa.fr



Équipe-projet CAIRN
<http://www.irisa.fr/cairn>

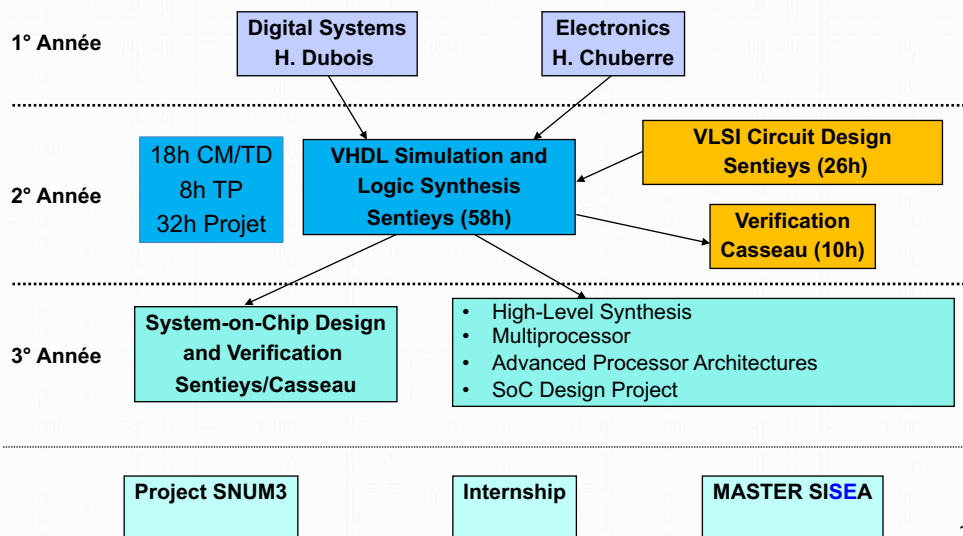


ENSSAT
LANNION
FUMIGNON



-1

Courses in Circuit Design at ENSSAT/SNUM

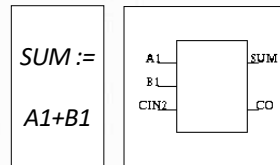


2

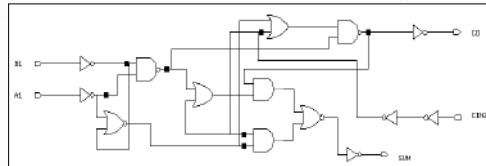
-2

Design Flow: from algorithm to circuit

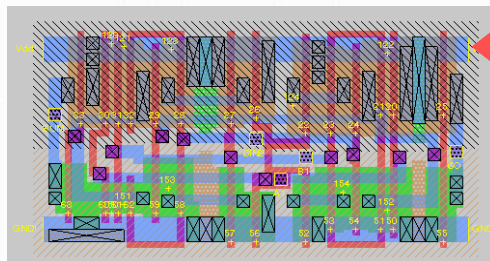
Register Transfer Level (RTL)



Gate Level

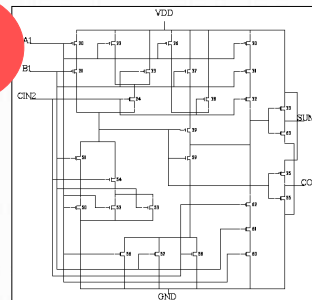


Algorithm



Layout Level

Circuit



Transistor or Circuit Level

3

-3

Hardware vs. Software

• Hardware

- Layout-Level
- Gate-Level (VHDL, Verilog)
- Register-Transfer Level (VHDL, Verilog)
- Algorithmic Level (C/C++, System Verilog)
- System Level (e.g. UML, Matlab/Simulink)

Abstraction Level



• Software

- Binary Code
- Assembly Code
- Machine Dependent Languages (e.g. C)
- Virtual Machine (e.g. Java)
- System Specifications (e.g. UML)

4

-4

Detailed Outline

1. Introduction: Why HDLs?
2. Design Flow and Tools
3. Basic Language Concepts ⚠
4. Signal and Delay Models
5. Modeling Digital Systems ⚠ ⚠ ⚠
6. Concurrent and Sequential Processes
 1. Process statement ⚠
 2. Process event behavior
 3. Signals vs. variables
 4. Timing behavior of processes ⚠
7. Modeling Structures
 1. Structural models ⚠
 2. Generics
 3. The Generate Statement
7. Simulation and Validation
 1. Concepts
 2. Writing testbenches ⚠ ⚠ ⚠
 3. Configurations
8. RTL and Logic Synthesis
 1. Writing Style for Logic Synthesis
 2. Combinational Logic
 3. Sequential Logic
 4. RTL and Logic Synthesis CAD Algorithms
9. Fil Rouge Example: FIR filter
⚠ ⚠ ⚠ ⚠ ⚠ ⚠ ⚠
10. Appendix
 1. Data Types
 2. Operators
 3. Attributes
 4. Subprograms, Packages, and Libraries
 5. Inputs and Outputs
 6. VHDL examples
 7. Synthesis hints

⚠: Simulation with ModelSim

5

-5

VHDL

- What is VHDL?
 - VHSIC (Very High Speed Integrated Circuit) **H**ardware **D**escription **L**anguage
- History
 - Designed by IBM, Texas Instruments, and Intermetrics as part of the DoD funded VHSIC program
 - Standardized by the IEEE in 1987: IEEE 1076-1987
 - Enhanced version of the language: IEEE 1076-1993
 - Additional standardized packages provide definitions of data types and expressions of timing data
 - IEEE 1164 (data types), IEEE 1076.3 (numeric), IEEE 1076.4 (timing)

6

-6

Programming vs. Hardware Description Languages

- Programming languages provide the *how*
 - for computation
 - for data manipulation
 - for execution on a specific hardware model
- HDL languages *describe* a system
 - Systems can be described from different points of view
 - Behavior: what does it do?
 - Structure: what is it composed of?
 - Functional properties: how to interface it?
 - Physical properties: how fast is it?

```
C Code
a=b+c;
d=a+e;
```

```
VHDL
a<=b+c;
d<=a+e;
```

7

-7

Usage

- Descriptions can be at different levels of abstraction
 - Switch level: model switching behavior of transistors
 - Gate level: model behavior of gates
 - Register transfer level (RTL): model combinational and sequential logic components
 - Functional Instruction Set Architecture level: functional behavior of a microprocessor
- Descriptions can be used for
 - Specification
 - Simulation
 - Synthesis

8

-8

Why do we Describe Systems?

- Design Specification
 - unambiguous definition of components and interfaces in a large design
- Design **Simulation**
 - verify system/subsystem/chip performance prior to design implementation
- Design **Synthesis**
 - automated generation of a hardware design

9

-9

VHDL: a First Example

- Interface
 - List of ports to connect to external components
 - Entity
- Behavior
 - Describes what the component is doing
 - Architecture

```
Entity and3 is  
  Port(  
    e1,e2,e3: in bit;  
    s: out bit);  
End and3;
```

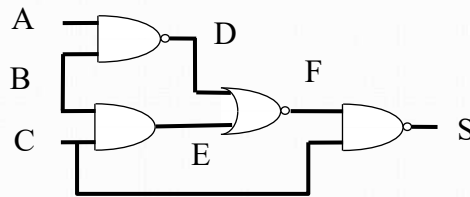
```
Architecture RTL of and3 is  
  Begin  
    s <= e1 and e2 and e3;  
  End [RTL];
```

10

-10

Abstraction Levels: Logic-Level

- Boolean equations



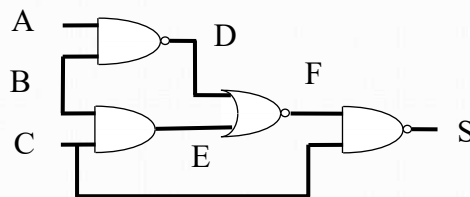
```
S<=not (F and C);  
F<=not (D or E);  
E<=B and C;  
D<=not (A and B)
```

11

-11

Abstraction Levels: Gate-Level

- Structural mapping with port map constructs



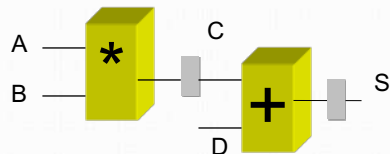
```
...  
U43 : NAND2LL port map (I0=>A, I1=>B, Z=>D);  
U44 : NOR2LL port map (I0=>D, I1=>E, Z=>F);  
...
```

12

-12

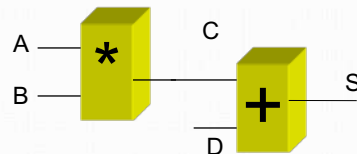
Abstraction Levels: Register Transfer Level

- Equation between registers or I/Os



```

Process (clk)
Begin
If clk'event and clk='1' then
    S<=D+C;
    C<=A*B;
End if;
End process;
    
```



```

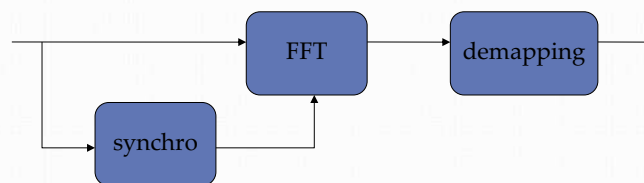
Process (A,B,C,D)
Begin
    S<=D+C;
    C<=A*B;
End process;
    
```

13

-13

Abstraction Levels: System-Level

- Structural mapping with function calls



```

FFT (clk, data_in, sync, fft_out);
Synchro (clk, data_in, sync);
Demapping (clk, fft_out, data_out);
    
```

- Same can be obtained with port map expressions
 - See later in the course

14

-14

Design Flow and Tools

15

-15

Execution Models for VHDL Programs

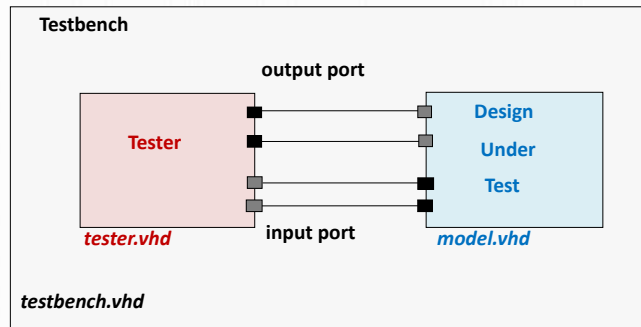
Two classes of execution models govern the application of VHDL programs

- For Simulation
 - Discrete event simulation
 - Understanding VHDL semantics is invaluable in coding and debugging programs
- For Synthesis
 - Hardware inference
 - Follows Register Transfer Level (RTL) semantics
 - The resulting circuit is a function of the building blocks used for implementation
 - Based on a gate library
 - Evaluates cost/performance

16

-16

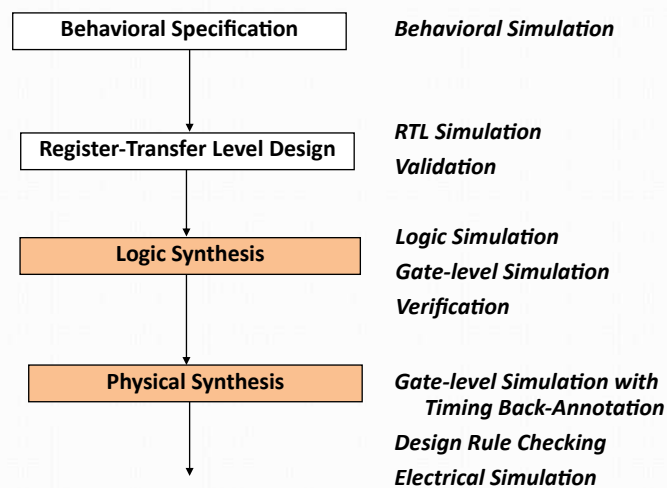
Simulation and Synthesis



17

-17

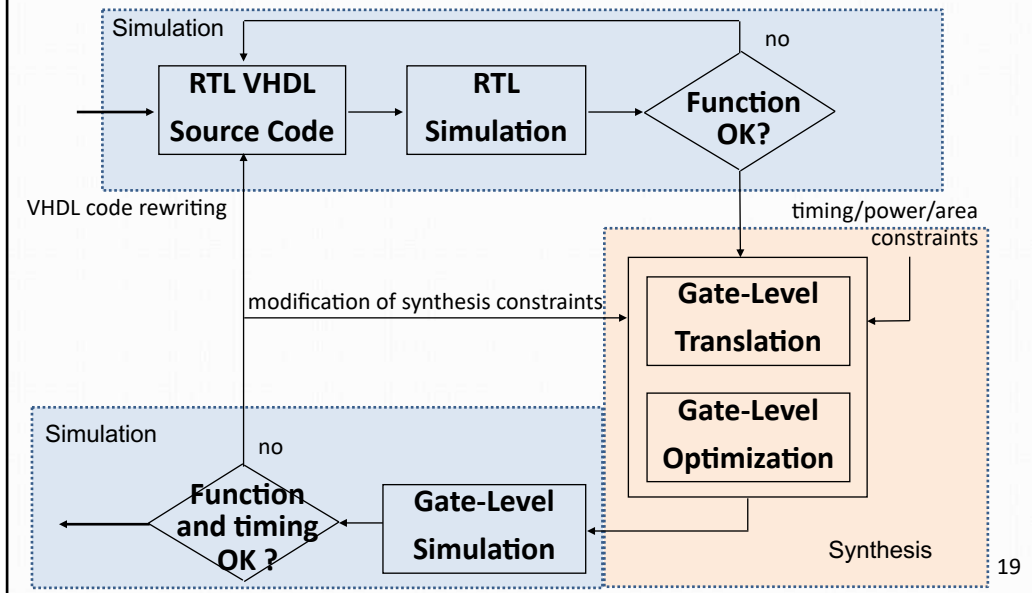
Design Flow



18

-18

Design Flow: Simulation and Synthesis



-19

CAD Tools

- Simulation
 - **Modelsim (Mentor Graphics)**
 - ISim (Xilinx)
 - Activ-HDL (Aldec)
 - Quartus Simulator (Altera)
 - (IES) Incisive Enterprise Simulator (Cadence)
 - VCS (Synopsys)
- Logic and RTL Synthesis
 - Xilinx XST (Vivado)
 - **Altera Quartus**
 - Leonardo Spectrum (Mentor Graphics)
 - **Design Compiler (Synopsys)**
- High-Level Synthesis (HLS)
 - Catapult-C, CtoS, Vivado HLS

20

-20

Other HDLs: Verilog

- Similar to VHDL

```
Entity and3 is
  Port(
    e1,e2,e3: in bit;
    s: out bit);
End and3;
Architecture RTL of and3 is
Begin
  s <= e1 and e2 and e3;
End RTL;
```

VHDL code

```
Module and3(e1,e2,e3);
  Input e1,e2,e3;
  Output s;

  Assign s= e1 & e2 & e3;
Endmodule
```

Verilog code

21

-21

Other HDLs: System C

- C++ classes to model HW (and SW)

```
#include "systemc.h"
SC_MODULE(and3) {
    sc_in<bool> e1;
    sc_in<bool> e2;
    sc_in<bool> e3;
    sc_out<bool> s;
    void compute_and() {
        s = e1 & e2 & e3;
    };
    SC_CTOR(and3) {
        SC_METHOD(compute_and);
        sensitive << e1 << e2 << e3;
    }
};
```

22

-22

Basic Language Concepts

23

-23

Layout of a VHDL Program

- Primary design units
 - Entity
 - Configuration
 - Package Declaration
- Secondary design units
 - Package body
 - Architecture

```
-- library declaration
...

-- Primary design units
...

-- Secondary design units
...
```

24

-24

Layout of a VHDL Program

- Usually, one file = one entity
 - test.vhd or test.vhdl
- Names, identifiers and variables
 - start by a letter
 - alphanumeric, '_'
 - case insensitive
 - do not end with "_"

Comments

```
-- library declaration
library IEEE;
use IEEE.std_logic_1164.all;
-- entity of component test
entity test is
    ...
end;
-- architecture B1 of test
architecture B1 of test
    ...
end;

architecture B_2 of Test
    ...
end;
```

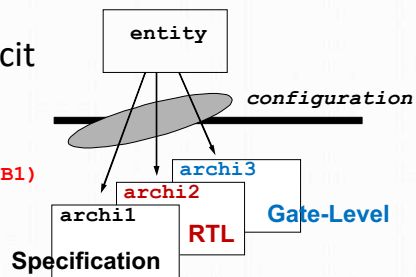
25

-25

Configurations

- Separate the specification of the interface from that of the implementation
 - An entity may have multiple architectures
- Configurations associate an entity with an architecture
 - Binding rules: default and explicit

```
Configuration conf1 of top is
For arch_top
  for instancel:test use entity work.test (B1)
  end for;
End for;
End conf1;
```



26

-26

Describing Design Entities and Architectures

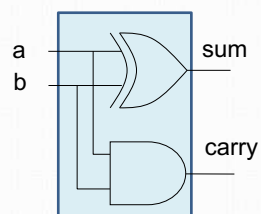
- Describing a digital system
 - Interface: how do we connect to it
 - Entity
 - Function: what does it do?
 - Architecture

27

-27

Describing the Interface: Entity

- The interface is a collection of ports
 - Ports are a new programming object: signal
 - Ports have a type: e.g., bit
 - Ports have a mode: in, out, inout (bidirectional), buffer (same as output but can be read)

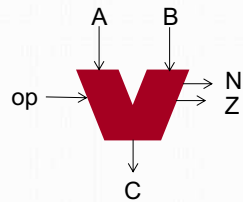


```
entity case insensitive half_ADDer is
port ( a, b : in bit;
      sum, carry: out bit );
end VHDL 1993 entity optional half_adder;
```

28

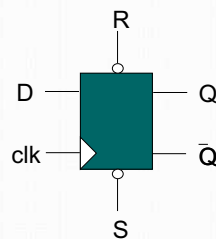
-28

Examples: Entity Descriptions



```
entity ALU32 is
port( A, B: in bit_vector (31 downto 0);
      C : out bit_vector (31 downto 0);
      Op: in bit_vector (5 downto 0);
      N, Z: out bit);
end ALU32;
```

↙ ↘
 MSB LSB

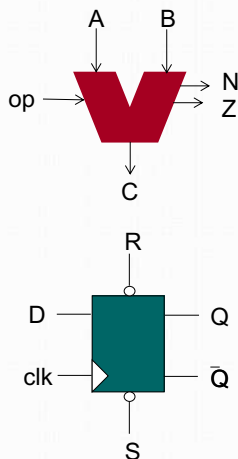


```
entity D_ff is
port( D, Q, Clk, R, S: in bit;
      Q, Qbar : out bit);
end D_ff;
```

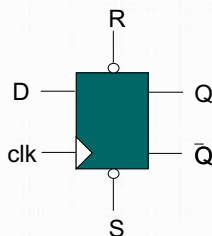
29

-29

Examples: Entity Descriptions (IEEE 1164)



```
entity ALU32 is
port( A, B: in std_logic_vector (31 downto 0);
      C : out std_logic_vector (31 downto 0);
      Op: in std_logic_vector (5 downto 0);
      N, Z: out std_logic);
end entity ALU32;
```



```
entity D_ff is
port( D, Q, Clk, R, S: in std_logic;
      Q, Qbar : out std_logic);
end entity D_ff;
```

Type `bit` is not powerful enough for realistic simulation: **use the IEEE 1164 type system**

30

-30

IEEE 1164 Value System

- `std_logic` Signal Values

Value	Interpretation
U	Uninitialized
X	Forcing Unknown
0	Forcing 0
1	Forcing 1
Z	High Impedance
W	Weak Unknown
L	Weak 0
H	Weak 1
-	Don't Care

31

-31

Port Objects OUT vs. BUFFER

```
entity HA is
  port(
    A,B,C: in std_logic;
    S1,S2: buffer std_logic);
end HA;

architecture RTL of HA is
begin
  S1<= A xor B;
  S2<= S1 and C;
end RTL;
```

```
entity HA is
  port(
    A,B,C: in std_logic;
    S1,S2: out std_logic );
end HA;

architecture RTL of HA is
  signal S1_i: std_logic;
begin
  S1_i <= A xor B;
  S2    <= S1_i and C;
  S1    <= S1_i;
end RTL;
```

32

-32

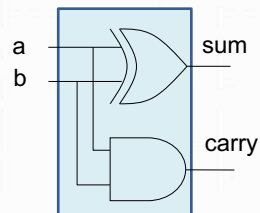
Describing Behavior: Architecture

- Description of events on output signals in terms of events on input signals
 - signal assignment statement
- Specification of propagation delays
 - if necessary

```
entity HalfAdder is
port (a, b : in bit;
      sum, carry :out bit);
end entity HalfAdder;

architecture behavioral of HalfAdder is
-- declarations
begin
sum <= (a xor b) after 5 ns;
carry <= (a and b) after 5 ns;
end architecture behavior;
```

VHDL 1993



33

-33

Describing Behavior: Architecture

- Use of the IEEE 1164 value system requires inclusion of the library and package declaration statements

```
library IEEE;
use IEEE.std_logic_1164.all;
```

} Declarations for a design entity

Types
Constant
Signals
Components
Functions
Procedures

```
entity HalfAdder is
port (a, b : in std_logic;
      sum, carry :out std_logic);
end HalfAdder;

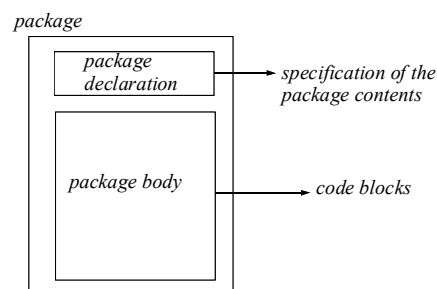
architecture behavioral of HalfAdder is
-- declarations
begin
sum <= (a xor b) after 5 ns;
carry <= (a and b) after 5 ns;
end behavioral;
```

34

-34

Libraries and Packages

- Libraries are logical units that are mapped to physical directories
- Packages are repositories for type definitions, procedures, and functions



35

-35

A VHDL Model Template

```

library library-name-1, library-name-2;
use library-name-1.package-name.all;
use library-name-2.package-name.all;

entity entity_name is
port(  input signals : in type;
        output signals : out type);
end entity_name;

architecture arch_name of entity_name is
  -- declare internal signals
  signal internal_sig1 : type := init;
  signal internal_sig2 : type := init;
  begin
  -- specify value of each signal as a function of other
  signals
  internal_sig1 <= simple, conditional, ...;
  internal_sig1 <= simple, conditional, ...;

  output_sig1  <= simple, conditional, ...;
  output_sig2  <= simple, conditional, ...;
end arch_name;
  
```

} Declare external libraries and visible components

} Define the interface

} Declare signals used to connect components

} Definition of how & when **internal** signal values are computed

} Definition of how & when **external** signal values are computed

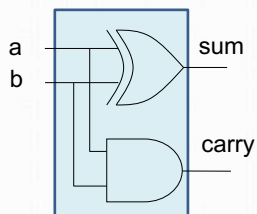
36

-36

Your First VHDL Design

⚠ simulate
this example
on vsim

- Launch Modelsim
 - vsim&
- File->New->Project
 - ~/VHDL
- Create New File
 - HalfAdder.vhdl
- Compile->Compile All



HalfAdder.vhdl

```
library IEEE;
use IEEE.std_logic_1164.all;

entity HalfAdder is
port (a, b : in std_logic;
      sum, carry :out std_logic);
end HalfAdder;

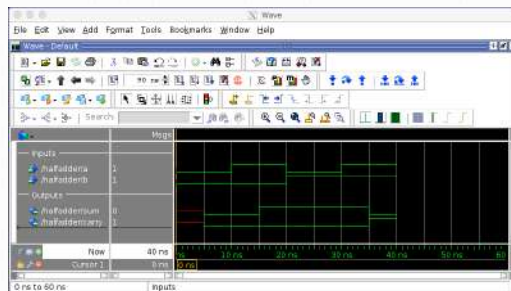
architecture behavioral of HalfAdder is
begin
    sum <= (a xor b) after 5 ns;
    carry <= (a and b) after 5 ns;
end behavioral;
```

37

-37

Your First VHDL Simulation

- File->New->Source->Do
- File->Save As
 - sim_HalfAdder.do
- Tools->Tcl->Execute Macro
 - sim_HalfAdder.do
 - (or do sim_HalfAdder.do)
- You should obtain this:



```
# Load the HalfAdder for simulation
vsim HalfAdder
# Open some selected windows for viewing
view structure
view signals
view wave
# Show some of the signals in the wave window
add wave -noupdate -divider -height 32 Inputs
add wave -noupdate a
add wave -noupdate b
add wave -noupdate -divider -height 32 Outputs
add wave -noupdate sum
add wave -noupdate carry
# Set some test patterns
# a = 0, b = 0 at 0 ns
force a 0 0
force b 0 0
# a = 1, b = 0 at 10 ns
force a 1 10
# a = 0, b = 1 at 20 ns
force a 0 20
force b 1 20
# a = 1, b = 1 at 30 ns
force a 1 30
# Run the simulation for 40 ns
run 40ns
```

sim_HalfAdder.do

38

-38

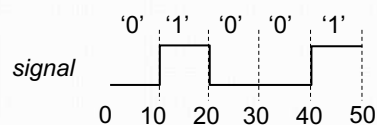
Signal Assignment and Delay Models

39

-39

The Signal Object Type

- VHDL supports four basic objects: variables, constants, signals and file types (see later)
- Variable and constant types
 - Follow traditional concepts
- Signal object type is motivated by digital system modeling
 - Distinct from variables in the **association of time with values**
 - Implementation of a signal is a **sequence of time-value pairs**



40

-40

Simple Signal Assignment Statement

- Use of `signals` in the architecture
 - Internal signals connect components
 - In the absence of initialization, default values are determined by signal type
- A statement is executed when an event takes place on a signal in the right-hand side of an expression
 - Textual order **does not imply** execution order
 - Order of statement execution **follows** propagation of events in the circuit

```
signal enable: std_logic := '0';  
...  
enable <= ...
```

41

-41

Constants and Alias



- Constant programming object
 - Values cannot be changed

```
constant IS_ZERO: std_logic := '0';  
...  
enable <= IS_ZERO
```

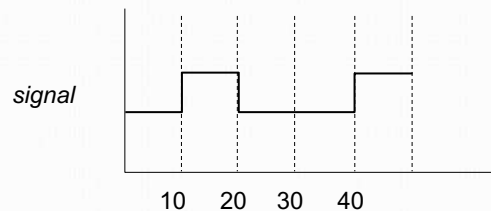
- Alias construct

```
signal ports: std_logic_vector(31 downto 0);  
alias command: std_logic is ports(28);
```

42

-42

Example: Waveform Generation



```
signal <= '0','1' after 10 ns,'0' after 20 ns,'1' after 40 ns;
```

- Multiple waveform elements can be specified in a single signal assignment statement
- Describe the signal transitions in time
 - Each transition is specified as a waveform element

43

-43

Conditional Signal Assignment (CSA)

```
library IEEE;
use IEEE.std_logic_1164.all;
entity mux4 is
port ( In0, In1, In2, In3 : in std_logic_vector (7 downto 0);
      Sel: in std_logic_vector(1 downto 0);
      Z : out std_logic_vector (7 downto 0));
end mux4;
architecture behavioral of mux4 is
begin
  Z <= In0 after 5 ns when Sel = "00" else
       In1 after 5 ns when Sel = "01" else
       In2 after 5 ns when Sel = "10" else
       In3 after 5 ns when Sel = "11" else
       "00000000" after 5 ns;
```

← Evaluation Order
is important!

- First true conditional expression determines the output value

44

-44

Unaffected Signals



- Value of the signal is not changed

```
library IEEE;
use IEEE.std_logic_1164.all;
entity pr_encoder is
port ( S0,S1,S2,S3: in std_logic;
      Z: out std_logic_vector (1 downto 0));
end pr_encoder;
architecture behavioral of pr_encoder is
begin
Z <=  "00" after 5 ns when S0 = '1' else
      "01" after 5 ns when S1 = '1' else
      unaffected when S2 = '1' else
      "11" after 5 ns when S3 = '1' else
      "00" after 5 ns;
end behavioral;
```

45

-45

Selected Signal Assignment Statement

```
entity mux4 is
port ( In0, In1, In2, In3: in std_logic_vector(7 downto 0);
      Sel: in std_logic_vector(1 downto 0);
      Z: out std_logic_vector(7 downto 0));
end mux4;
architecture behavioral-2 of mux4 is
begin
with Sel select
  Z <=  (In0 after 5 ns) when "00",
        (In1 after 5 ns) when "01",
        (In2 after 5 ns) when "10",
        (In3 after 5 ns) when "11"
        (In3 after 5 ns) when others;
end behavioral-2;
```

All options must be covered
and only one must be true!

- The "when others" clause can be used to ensure that all options are covered
- The "unaffected" clause may also be used here

46

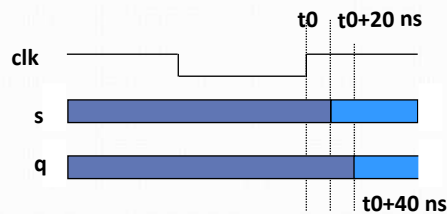
-46

Guarded Blocks

```
[label:] block (optional_guard_condition)
[declarations]
begin
    concurrent statements
end block label;
```

- Merge concurrent instructions in a single block
 - Share declarations
 - Guard assignment with a condition

```
B1 : block (clk='1' and not clk'stable)
begin
    s <= guarded value1 after 20 ns;
    q <= guarded value2 after 40 ns;
end block B1
```



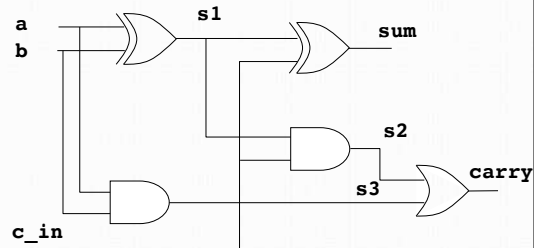
47

-47

Simple Signal Assignment

```
library IEEE;
use IEEE.std_logic_1164.all;
entity FullAdder is
port (a, b, c_in: in std_logic;
      sum, carry: out std_logic);
end FullAdder;
```

```
architecture dataflow of FullAdder is
    signal s1, s2, s3 : std_logic;
    constant gate_delay: time := 5 ns;
begin
    L1: s1 <= (a xor b) after gate_delay;
    L2: s2 <= (c_in and s1) after gate_delay;
    L3: s3 <= (a and b) after gate_delay;
    L4: sum <= (s1 xor c_in) after gate_delay;
    L5: carry <= (s2 or s3) after gate_delay;
end dataflow;
```



⚠ simulate
this example
on vsim

48

-48

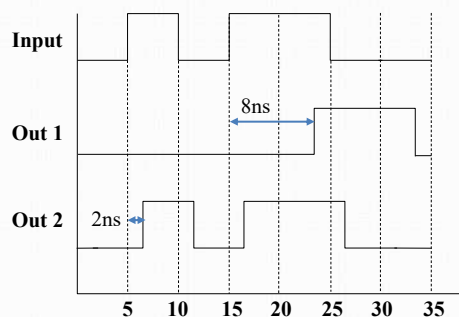
Delay Models in VHDL

- Inertial delay
 - Default delay model
 - Suitable for modeling delays through devices such as **gates**
- Transport Delay
 - Model delays through devices with very small inertia such as **wires**
 - All input events are propagated to output signals
- **Delta (Δ) delay**
 - What about models where no propagation delays are specified?
 - Infinitesimally small delay is automatically inserted by the simulator to preserve correct ordering of events

49

–49

Inertial Delays: Example



`Out1 <= inertial Input after 8 ns`
signal value Input is assigned to the signal Out1 with 8 ns delay
if the width of an impulse is shorter than 8 ns then it will not be transmitted

`Out2 <= inertial Input after 2 ns`

`Out1 <= reject 6 ns`
`inertial Input after 8 ns`

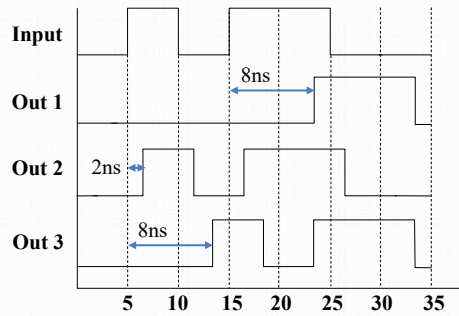
```
signal <= reject time_expression inertial value_expression
after time_expression;
```

- Most general form of a waveform element
- Suitable for modeling **gate delay**

50

–50

Transport Delays: Example



Out1 <= Input **after** 8 ns

Out2 <= **inertial** Input **after** 2 ns

Out3 <= **transport** Input **after** 8 ns

signal <= transport value_expression after time_expression;

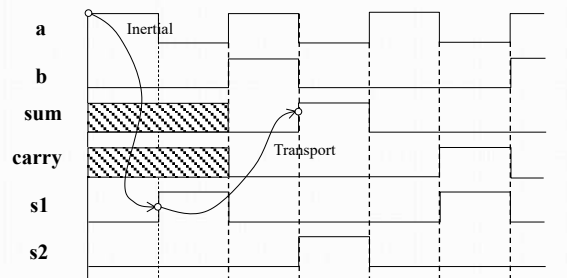
- Suitable for modeling **wire delay**

51

-51

Delays: Example

```
architecture transport_delay of half_adder is
    signal s1, s2: std_logic:= '0';
begin
    s1 <= (a xor b) after 2 ns;
    s2 <= (a and b) after 2 ns;
    sum <= transport s1 after 4 ns;
    carry <= transport s2 after 4 ns;
end architecture transport_delay;
```



52

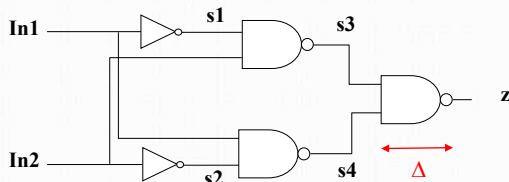
-52

Delta Delays: Example

```
architecture behavior of combinational
  signal s1, s2, s3, s4: std_logic:= '0';
begin
  s1 <= not In1;
  s2 <= not In2;
  s3 <= not (s1 and In2);
  s4 <= not (s2 and In1);
  z <= not (s3 and s4);
end architecture behavior;
```

- Equivalent to

```
z <= not (s3 and s4) after Δ;
s4 <= not (s2 and In1) after Δ;
s3 <= not (s1 and In2) after Δ;
s2 <= not In2 after Δ;
s1 <= not In1 after Δ;
```



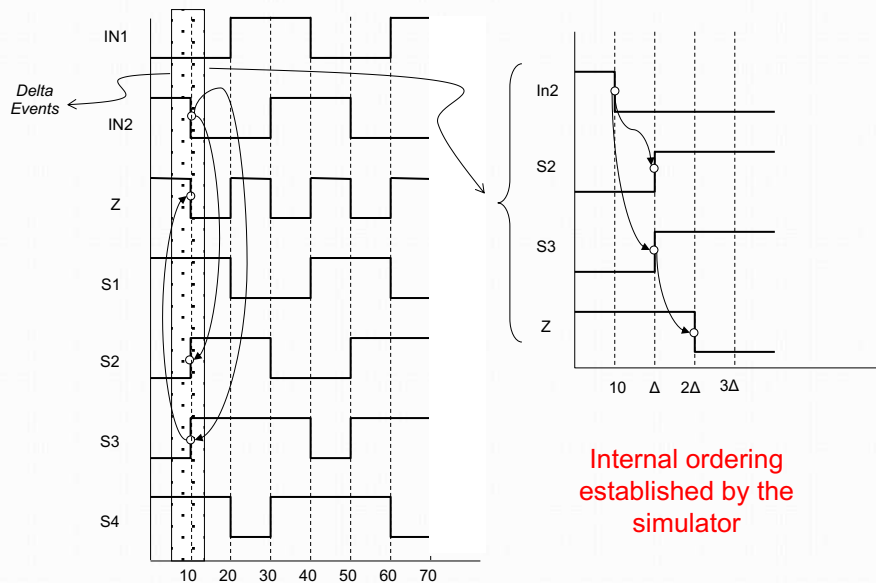
- Reverse order is equivalent

```
z <= not (s3 and s4);
s4 <= not (s2 and In1);
s3 <= not (s1 and In2);
s2 <= not In2;
s1 <= not In1;
```

53

-53

Delta Delays: Behavior



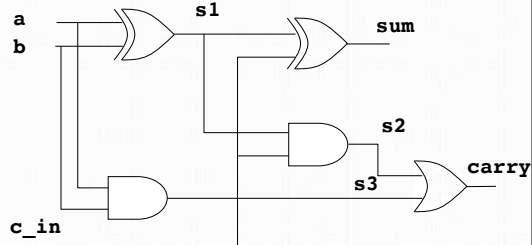
54

-54

Simple Signal Assignment

```
library IEEE;
use IEEE.std_logic_1164.all;
entity FullAdder is
port (a, b, c_in: in std_logic;
      sum, carry: out std_logic );
end FullAdder;
```

```
architecture dataflow of FullAdder is
    signal s1, s2, s3 : std_logic;
    constant gate_delay: time:= 15 ns;
begin
    L1: s1 <= (a xor b);
    L2: s2 <= (c_in and s1) after 5 ns;
    L3: s3 <= inertial (a and b) after 5 ns;
    L4: sum <= (s1 xor c_in) after gate_delay;
    L5: carry <= transport (s2 or s3) after gate_delay;
end dataflow;
```



! simulate
this example
on vsim

55

-55

Simulate This Example (Ex1)

sim_Ex1.do

```
vsim Ex1
view wave
add wave -noupdate a
add wave -noupdate b
force a 0 0
force b 1 0
run 40
```

Ex1.vhdl

```
entity Ex1 is
end Ex1;

architecture Ex1 of Ex1 is
    signal A, B: bit := '0';
begin
    A <= not B after 5 ns;
    B <= A after 5 ns;
end Ex1;
```

- What is the problem?
 - Event-driven simulation

56

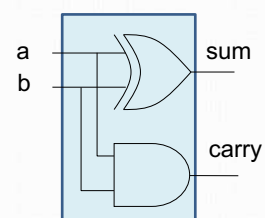
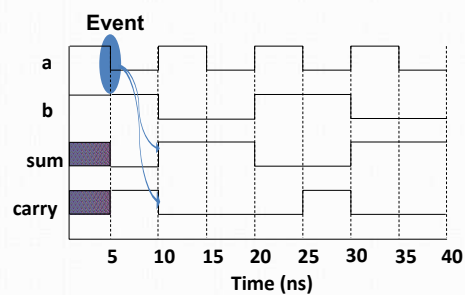
-56

Modeling and Simulating Digital Systems

57

-57

Attributes of Digital Systems



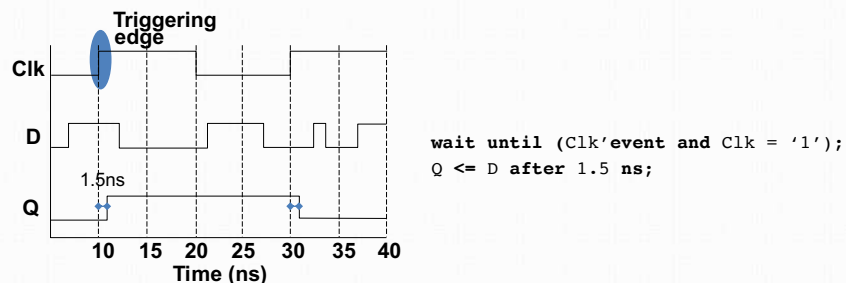
```
sum <= (a xor b) after 5 ns;  
carry <= (a and b) after 5 ns;
```

- Digital systems are about signals and their values
 - Signal value changes at specific points in time
 - Time ordered sequence of **events** produces a waveform

58

-58

Attributes of Digital Systems



- Timing: computation of events takes place at specific points in time
- Need to “wait for” an event: in this case the clock

59

-59

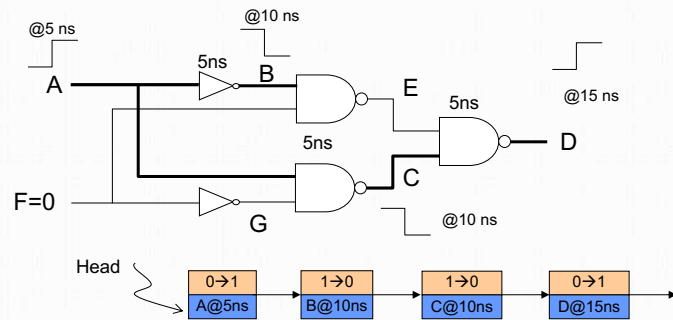
Discrete Event Simulation

- Digital systems are modeled as the generation of events (value transitions) on signals
- Discrete event simulations **manage the generation and ordering of events**
 - Correct sequencing of event processing
 - Correct sequencing of computations caused by events
- Two-step model of the progression of time
 - Evaluate all affected components at the current time: events on input signals
 - Schedule future events and move to the next time step: the next time at which events take place

60

-60

Simulation of Digital Systems

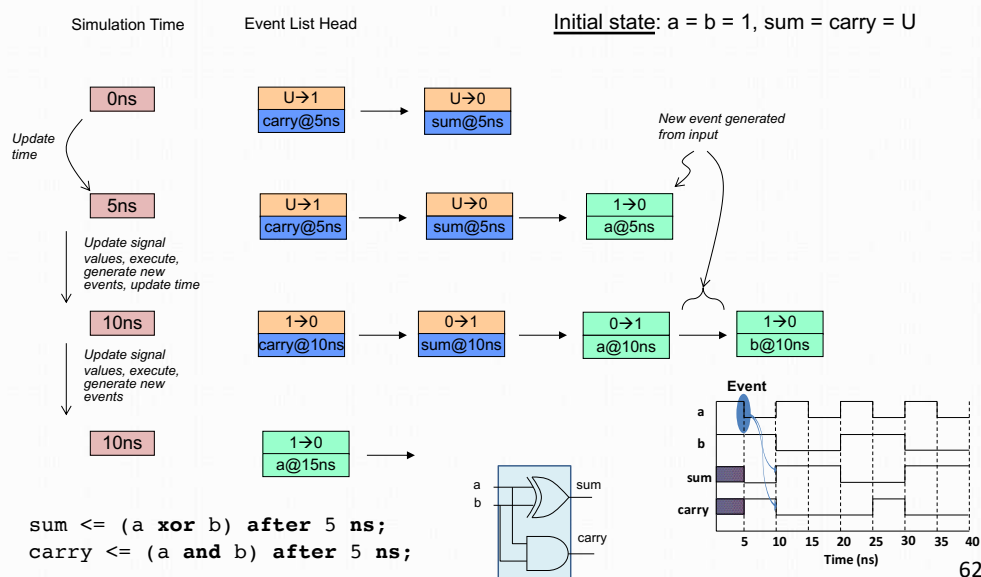


```
D <= (E nand C) after 5 ns;
E <= (B nand F) after 5 ns;
C <= (A nand G) after 5 ns;
G <= (not F) after 5 ns;
B <= (not A) after 5 ns;
```

61

-61

Discrete Event Simulation: Example



62

-62

Back to Example One (Ex1)

sim_Ex1.do

```
vsim Ex1
view wave
add wave -noupdate a
add wave -noupdate b
force a 0 0
force b 1 0
run 40
```

Ex1.vhdl

```
entity Ex1 is
end Ex1;

architecture Ex1 of Ex1 is
    signal A, B: bit := '0';
begin
    A <= not B after 5 ns;
    B <= A after 5 ns;
end Ex1;
```

- How to solve the problem?
 - Concurrent processes

63

-63

Modeling Concurrent and Sequential Processes

64

-64

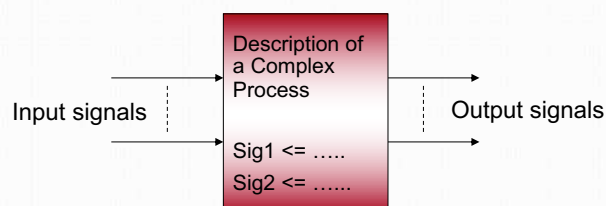
Raising the Level of Abstraction

- Concurrent signal assignment statements can easily capture the gate level behavior of digital systems
- Higher level digital components have more complex behaviors
 - Input/output behavior not easily captured by concurrent signal assignment statements
 - Models utilize state information
 - Incorporate data structures
- We need more powerful constructs
- Examples: memory, processor, state machines

65

–65

Extending the Event Model



- Combinational logic input/output semantics
 - Events on inputs causes re-computation
 - Re-computation may lead to events on outputs
- Computation of the value and time of output events can be a complex process

66

–66

The Process Statement

```
entity mux4 is
port ( In0, In1, In2, In3: in std_logic_vector (7 downto 0);
      Sel: in std_logic_vector(1 downto 0);
      Z : out std_logic_vector (7 downto 0));
end entity mux4;

architecture behavioral-3 of mux4 is
begin
  process (Sensitivity List Sel, In0, In1, In2, In3) is
    Use of variables rather than signals
    variable Zout: std_logic;
  begin
    if (Sel = "00") then Zout := In0;
    elsif (Sel = "01") then Zout := In1;
    elsif (Sel = "10") then Zout := In2;
    else Zout:= In3;
    end if;
    Z <= Zout; } Signal Assignment
  end process;
```

67

-67

The Process Construct

- A process is executed when a event in the sensitivity list occurs
- Statements in a process are executed sequentially
- A process body is structured much like conventional C function
 - Declaration and use of variables
 - if-then, if-then-else, case, for and while constructs
 - A process can contain signal assignment statements
- A process executes concurrently with other concurrent signal assignment statements or processes
- A process takes 0 seconds of simulated time to execute and may schedule events in the future

68

-68

Inside the Process: Sequential Constructs

- **if and case**

```

case data is
  when "001" to "011"      => nbBit<="01";
  when "100"|"110"|"101"   => nbBit<="10";
  when "111"                => nbBit<="11";
  when others               => nbBit<="00";
end case;

```

```

if a='1' then
  c<= "010";
elsif b='1' then
  c<= "101";
else
  c<= "000";
end if;

```

```

case state is
  when idle =>
    if sel='0' then a<= "101";
    else a<= "001";
    end if;
    ...
  when others => a<="000";
end case ;

```

69

-69

Inside the Process: Iteration

- **for loop** [label:] **for** var **in** min **to** max **loop**
 – Implicit declaration ...
end loop [label];
 - local to the loop
 - **while loop** [label:] **while** condition **loop**
 – Boolean expression for ...
end loop [label]; termination
 - **loop** [label:] **loop**
...
 - **next / exit** [label:] **exit** [label2] [**when** condition];
...
end loop [label];
- [label:] **next** [label2] [**when** condition];

70

-70

Inside the Process: Iteration

- Example: A Simple Multiplier

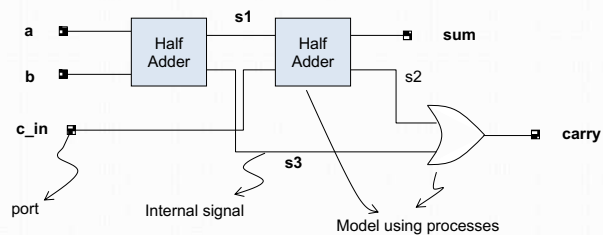
```
mult_process: process(multiplicand,multiplier) is
  variable product_register : std_logic_vector (63 downto 0) := X"0000000000000000";
  variable multiplicand_register : std_logic_vector (31 downto 0) := X"00000000";

begin
  multiplicand_register := multiplicand;
  product_register(63 downto 0) := X"00000000" & multiplier;
  for index in 1 to 32 loop
    if product_register(0) = '1' then
      product_register(63 downto 32) := product_register(63 downto 32) +
        multiplicand_register(31 downto 0);
    end if;
    -- perform a right shift with zero fill
    product_register (63 downto 0) := '0' & product_register (63 downto 1);
  end loop;
  -- write result to output port
  product <= product_register after module_delay;
end process mult_process;
```

71

-71

Concurrent Processes: Full Adder



- Each of the components of the full adder can be modeled using a process
- Processes execute concurrently
 - In this sense they behave exactly like concurrent signal assignment statements
- Processes communicate via signals

72

-72

Process Behavior

- All processes are executed once at start-up
- Thereafter dependencies between signal values and events on these signals determine process initiation
- One can view processes as components with an interface/function
- **Note that signals behave differently from variables!**

```
library IEEE;
use IEEE.std_logic_1164.all;
entity sig_var is
    port (x, y, z: in std_logic;
          res1, res2: out std_logic);
end sig_var;

architecture behavior of sig_var is
    signal sig_s1, sig_s2: std_logic;
    begin
        proc1: process (x, y, z) ...
        proc2: process (x, y, z) ...
    end behavior;
```

73

-73

Processes + Conditional Signal Assignments

- A process can be viewed as single concurrent signal assignment statement
 - The external behavior is the same as a CSA
 - Processes describe more complex event generation behavior
- Processes execute concurrently in simulated time with other CSAs

```
architecture behavioral of HalfAdder is
begin
    xor1: process (a, b) is
    begin
        sum <= (a xor b) after 5 ns;
    end process xor1;
    carry <= (a and b) after 5 ns; } Signal Assignment
end behavioral;
```

74

-74

Concurrent Processes: Full Adder

```
library IEEE;
use IEEE.std_logic_1164.all;
entity FullAdder is
    port (a, b, c_in: in std_logic;
          sum, carry: out std_logic);
end entity FullAdder;

architecture behavioral of FullAdder is
    signal s1, s2, s3: std_logic := '0';
    constant delay: time:= 5 ns;
begin

    HA1: process (a, b) is
    begin
        s1 <= (a xor b) after delay;
        s3 <= (a and b) after delay;
    end process HA1;

    HA2: process(s1, c_in) is
    begin
        sum <= (s1 xor c_in) after delay;
        s2 <= (s1 and c_in) after delay;
    end process HA2;

    OR1: process (s2, s3)
    begin
        carry <= (s2 or s3) after delay;
    end process OR1;

end behavioral;
```

 simulate
this example
on vsim

Need to fill the file FullAdder_process.vhd and to modify the do file to point on this architecture

75

-75

Signals vs. Variables

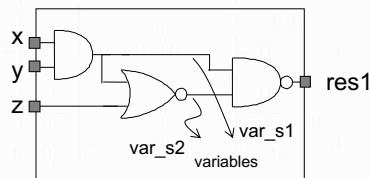
76

-76

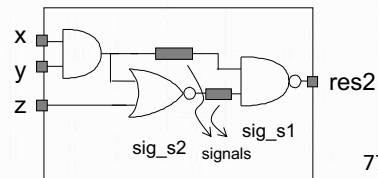
Variables vs. Signals: Example

- Distinction between the use of variables vs. signals
 - Computing values only vs. computing time-value pairs
 - Remember event ordering and delta delays!

```
proc1: process (x, y, z) is
    variable var_s1, var_s2: std_logic;
begin
    var_s1 := x and y;
    var_s2 := var_s1 xor z;
    res1 <= var_s1 nand var_s2 after 2 ns;
end process;
```



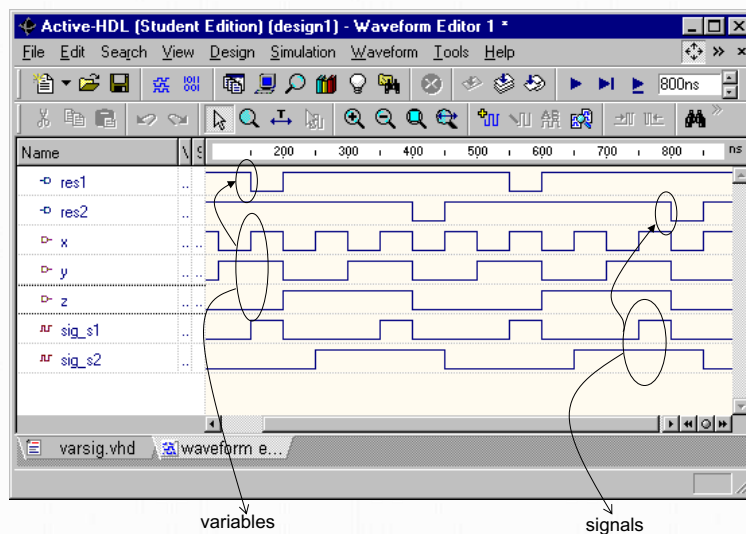
```
proc2: process (x, y, z)
begin
    sig_s1 <= x and y after 2 ns;
    sig_s2 <= sig_s1 xor z after 2 ns;
    res2 <= sig_s1 nand sig_s2 after 2 ns;
end process;
```



77

-77

Variables vs. Signals: Example



res1 (variables)
res2 (signals)

*This transition is
determined by process
initiation*

78

-78

Variables vs. Signals

- Writing processes
 - Use signals to represent corresponding hardware entities
 - Use variables when computing (future) values of signals

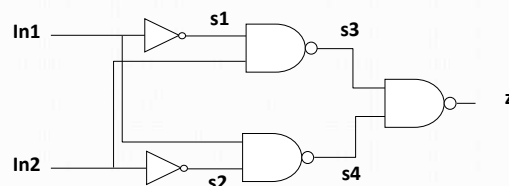
79

-79

Using Signals in a Process



- Signals are visible in a process
- Processes can encapsulate variable and signal assignment statements
- What is the effect on the model behavior between dataflow and process models?
- Actual waveforms will depend on how initialization is handled/performed



80

-80

Using Signals in a Process



```

library IEEE;
use IEEE.std_logic_1164.all;
entity combinational is
    port (In1, In2: in std_logic;
          z : out std_logic);
end combinational;
signal s1, s2, s3, s4: std_logic;
begin
    s1 <= not In1;
    s2 <= not In2;
    s3 <= not (s1 and In2);
    s4 <= not (s2 and In1);
    z <= not (s3 and s4);
end behavior;

```

*Encapsulate
in a process*

```

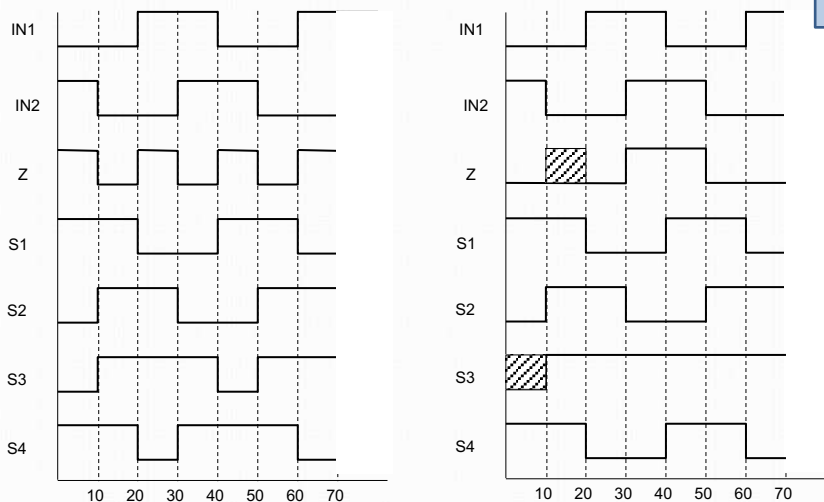
library IEEE;
use IEEE.std_logic_1164.all;
entity combinational is
    port (In1, In2: in std_logic;
          z : out std_logic);
end combinational;
signal s1, s2, s3, s4: std_logic;
begin
    sig_in_proc: process(In1, In2) is
    begin
        s1 <= not In1;
        s2 <= not In2;
        s3 <= not (s1 and In2);
        s4 <= not (s2 and In1);
        z <= not (s3 and s4);
    end process sig_in_proc;
end behavior;

```

81

-81

Using Signals in a Process (cont.)



Using concurrent signal assignment statements

Using signal assignment statements within a process

82

-82

Timing Behavior of Processes: the Wait Statement

83

-83

The Wait Statement

- The wait statements can describe synchronous or asynchronous timing operations

```
library IEEE;
use IEEE.std_logic_1164.all;
entity dff is
port (D, Clk : in std_logic;
      Q, Qbar : out std_logic);
end dff;
architecture behavioral of dff is
begin
  output: process is
  begin
    wait until (Clk'event and Clk = '1'); -- wait for rising edge
    Q <= D after 5 ns;
    Qbar <= not D after 5 ns;
  end process output;
end behavioral;
```

No sensitivity list

Means a value change on signal clk

84

-84

The Wait Statement

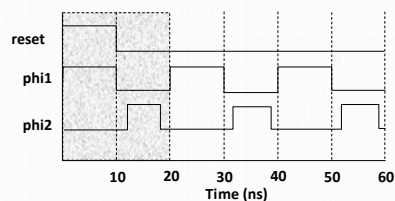
- A process can have multiple wait statements
- A process cannot have both a wait statement and a sensitivity list (it should have one or the other): why?
- wait statements provide explicit control over suspension and resumption of processes
 - Representation of both synchronous and asynchronous events in a digital systems

85

-85

The Wait Statement: Waveform Generation

```
library IEEE;
use IEEE.std_logic_1164.all;
entity two_phase is
    port(phi1, phi2, reset: out std_logic);
end two_phase;
architecture behavioral of two_phase is
begin
    rproc: reset <= '1', '0' after 10 ns;
    clock_process: process is
    begin
        phi1 <= '1', '0' after 10 ns;
        phi2 <= '0', '1' after 12 ns, '0' after 18 ns;
        wait for 20 ns;
    end process clock_process;
end behavioral;
```



```
clock_process: process is
begin
    [while true loop]
        phi1 <= '1';
        wait for 10 ns;
        phi1 <= '0';
        wait for 10 ns;
    [end loop;]
end process;
```

- Note the “perpetual” behavior of processes

86

-86

Replace Wait Statement with Events

```
entity asynch_dff is
port (R, S, D, Clk: in std_logic; Q, Qbar: out std_logic);
end asynch_dff;
architecture behavioral of asynch_dff is
begin
output: process (R, S, Clk) is
    execute on event on any signal
begin
    if (R = '0') then
        Q <= '0' after 5 ns;
        Qbar <= '1' after 5 ns;
    elsif S = '0' then
        Q <= '1' after 5 ns;
        Qbar <= '0' after 5 ns;
    elsif (Clk'event and Clk = '1') then
        synchronous clock
        Q <= D after 5 ns;
        Qbar <= (not D) after 5 ns;
    end if;
end process output;
end behavioral;
```

implied ordering provides asynchronous set and reset

87

-87

Back to Example One (Ex1)

sim_Ex2.do

```
vsim Ex2
view wave
add wave -noupdate a
run 40
```

Ex2.vhdl

```
entity Ex2 is
end Ex2;
architecture Ex2 of Ex2 is
    signal A: bit := '0';
begin
    process is
        variable B: bit := '0';
    begin
        ... -- ???
    end process;
end Ex2;
```

88

-88

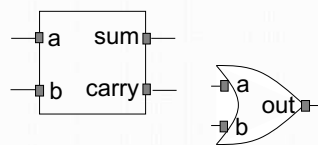
Modeling Structures

90

–90

Structural Models

- Structural models describe a digital system as an interconnection of components
 - Define the components used in the design
 - Describe the interconnection of these components
- Descriptions of the behavior of the components must be independently available as structural or behavioral models
 - An entity/architecture for each component must be available



91

–91

Structural Models

```

architecture structural of FullAdder is
  component HalfAdder is -- declaration of components used
    port (a, b: in std_logic;
          sum, carry: out std_logic);
  end component HalfAdder;
  component OR2 is
    port(a, b : in std_logic;
          c : out std_logic);
  end component or_2;
  signal s1, s2, s3: std_logic;
begin
  H1: HalfAdder port map (a=>In1, b=>In2, sum=>s1, carry=>s3);
  H2: HalfAdder port map (a=>s1, b=>c_in, sum=>sum, carry=>s2);
  O1: OR2 port map (b=>s3, c=>c_out, a=>s2);
end structural;

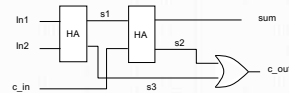
```

interconnection of the component ports

unique name of the components

connection

component instantiation statement



92

-92

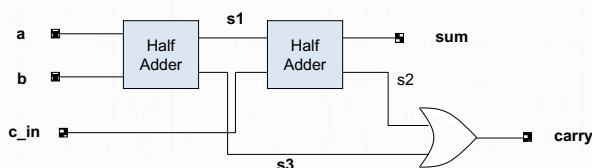
Structural Models: Full Adder

```

library IEEE;
use IEEE.std_logic_1164.all;
entity FullAdder is
  port (a, c_in, b: in std_logic;
        sum, carry: out std_logic);
end FullAdder;

architecture structural of FullAdder is
  component HalfAdder is
    port (a, b: in std_logic;
          sum, carry: out std_logic);
  end component HalfAdder;
begin
  H1: HalfAdder port map (a=>a, b=>b,
    sum=>s1, carry=>s3);
  H2: HalfAdder port map (s1, c_in,
    sum, s2);
  carry <= s2 OR s3;
end structural;

```



! simulate
this example
on vsim

93

-93

Hierarchy and Abstraction



- Structural descriptions can be nested
- The half adder may itself be a structural model

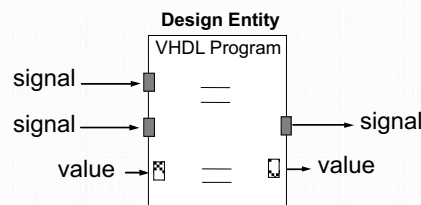
```
architecture structural of half_adder is
  component xor2 is
    port (a, b : in std_logic;
          c : out std_logic);
  end component xor2;
  component and2 is
    port (a, b : in std_logic;
          c : out std_logic);
  end component and2;
begin
  EX1: xor2 port map (a => a, b => b, c => sum);
  AND1: and2 port map (a => a, b => b, c => carry);
end architecture structural;
```

94

-94

Generics: Properties

- Generics are constant objects and can only be read
- The values of generics must be known at compile time
- They are a part of the interface specification but do not have a physical interpretation
- **Use of generics are a very powerful structuring mechanism**



95

-95

Generics

- Enables the construction of parameterized models

```
library IEEE;
use IEEE.std_logic_1164.all;

entity xor2 is
  generic (gate_delay : Time:= 2 ns);
  port(In1, In2 : in std_logic;
        z : out std_logic);
end xor2;

architecture behavioral of xor2 is
begin
  z <= (In1 xor In2) after gate_delay;
end behavioral;
```

96

-96

Generics in Hierarchical Models

- Parameter values are passed through the hierarchy

```
architecture generic_delay of half_adder is
  component xor2
    generic (gate_delay: Time);
    port (a, b : in std_logic;
          c : out std_logic);
  end component;
  component and2
    generic (gate_delay: Time);
    port (a, b : in std_logic;
          c : out std_logic);
  end component;
begin
  EX1: xor2      generic map (gate_delay => 6 ns)
                 port map(a => a, b => b, c => sum);
  A1: and2       generic map (gate_delay => 3 ns)
                 port map(a=> a, b=> b, c=> carry);
end generic_delay;
```

97

-97

Example: N-Input Gate

- Map the generics to create different size OR gates

```
entity generic_or is
  generic (n: positive:=2);
  port (in1: in std_logic_vector((n-1) downto 0);
        z: out std_logic);
end generic_or;
architecture behavioral of generic_or is
begin
  process (in1) is
    variable sum : std_logic:= '0';
  begin
    sum := '0';
    for i in 0 to (n-1) loop
      sum := sum or in1(i);
    end loop;
    z <= sum;
  end process;
end behavioral;
```

98

-98

Example: Using the Generic N-Input OR Gate

- Full adder model can be modified to use the generic OR gate model via the generic map () construct
- Analogy with macros

```
architecture structural of FullAdder is
  component generic_or
    generic (n: positive);
    port (in1 : in std_logic_vector ((n-1) downto 0);
          z : out std_logic);
  end component;
  ...
begin
  H1: HalfAdder port map (a=>In1, b=>In2, sum=>s1, carry=>s3);
  H2: HalfAdder port map (a=>s1, b=>c_in, sum=>sum, carry=>s2);
  O1: generic_or generic map (n=>2)
    port map (a=>s2, b=>s3, c=>c_out);
end structural;
```

99

-99



The Generate Statement

100

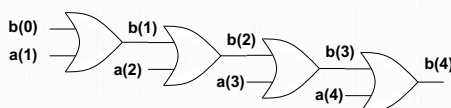
-100



The Generate Statement

- What if we need to instantiate a large number of components in a regular pattern?
 - Need conciseness of description
 - Iteration construct for instantiating components!
- The generate statement
 - A parameterized approach to describing the regular interconnection of components

```
a1: for i in 1 to 4 generate  
    a2: OR2 port map(b(i-1), a(i), b(i));  
end generate;
```



101

-101

Using the Generate Statement



- Identify components with regular interconnect
- Declare local arrays of signals for the regular interconnections
- Write the generate statement
 - Analogy with loops and multidimensional arrays
 - Beware of unconnected signals!
- Instantiate remaining components of the design

102

-102

The Generate Statement: Example



- Instantiating interconnected components
 - Declare local signals used for the interconnect

```
entity dregister is
port ( d : in std_logic_vector(7 downto 0);
      q : out std_logic_vector(7 downto 0);
      clk : in std_logic);
end entity dregisters
architecture structural of dregister is
begin
  dreg: for i in d'range generate
    reg: dff port map(d=>d(i), q=>q(i), clk=>clk);
  end generate;
end architecture structural;
```

103

-103

The Generate Statement: Example



```

library IEEE;
use IEEE.std_logic_1164.all;

entity multi_bit_generate is
  generic(gate_delay:time:= 1 ns;
    width:natural:=8); -- the default is a 8-bit ALU
  port( in1 : in std_logic_vector(width-1 downto 0);
    in2 : in std_logic_vector(width-1 downto 0);
    result : out std_logic_vector(width-1 downto 0);
    opcode : in std_logic_vector(1 downto 0);
    cin : in std_logic;
    cout : out std_logic);
end entity multi_bit_generate;

architecture behavioral of multi_bit_generate is

  component one_bit is -- declare the single bit ALU
    generic (gate_delay:time);
    port (in1, in2, cin : in std_logic;
      result, cout : out std_logic;
      opcode : in std_logic_vector (1 downto 0));
  end component one_bit;

  signal carry_vector: std_logic_vector(width-2 downto 0);
  -- the set of signals for the ripple carry

begin
  a0: one_bit generic map (gate_delay)
    -- instantiate ALU for bit position 0
    port map (in1=>in1(0), in2=>in2(0), result=>result(0), cin=>cin,
      opcode=>opcode, cout=>carry_vector(0));

  a2to6: for i in 1 to width-2 generate
    -- generate instantiations for bit positions 2-6
    a1: one_bit generic map (gate_delay)
      port map(in1=>in1(i), in2=> in2(i), cin=>carry_vector(i-1),
        result=>result(i), cout=>carry_vector(i),opcode=>opcode);
  end generate;

  a7: one_bit generic map (gate_delay)
    -- instantiate ALU for bit position 7
    port map (in1=>in1(width-1), in2=>in2(width-1), result=>
      result(width-1), cin=>carry_vector(width-2), opcode=>opcode,
      cout=>cout);
end architecture behavioral;

```

104

-104

Simulation and Validation

105

-105

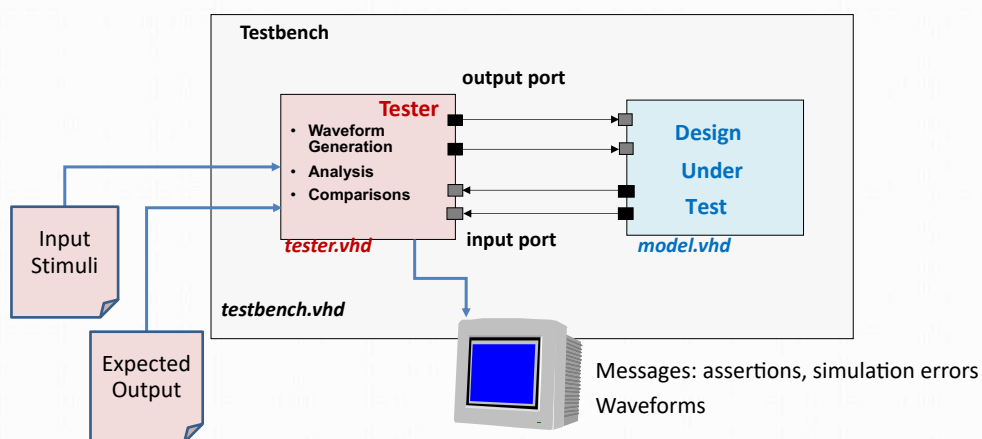
Simulation and Validation

- What is a simulation ?
 - Instantiation of the component to be tested
 - *Design Under Test (DUT)*
 - Writing a *TestBench* to simulate and validate the DUT
 - Initialization of input signals
 - **All input signals!**
 - Waveform as input stimuli
 - write a process to assign input signals (including clock and reset)
 - can be read from a file with input test vectors
 - Analysis of results from output transition
 - print (assert) errors and warning from a (pre-)calculated reference

106

–106

Simulation and Validation



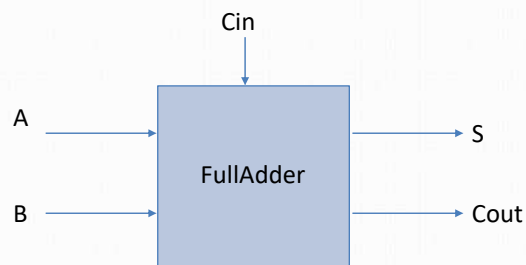
107

–107

Simulation and Validation

- Test Entity
 - Example: DUT = Full Adder

```
entity FullAdder is
    port (a, b, c_in: in std_logic ;
          sum, carry: out std_logic );
end FullAdder;
architecture behavioral of FullAdder is
    ...
end behavioral ;
```



108

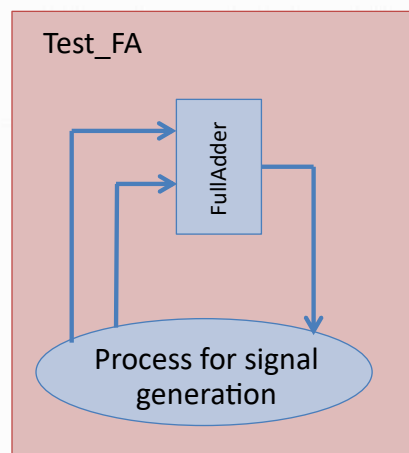
-108

Simulation and Validation

- Test Entity: instantiate a FullAdder component
 - Test_Add entity without port and generic

```
library IEEE;
use IEEE.std_logic_1164.all;
```

```
entity Test_FA is
end Test_FA;
```



109

-109

Simulation and Validation

– Test Architecture

- Component to be tested
- I/O signal declarations
- Instantiation of component
- Process generating input stimuli

```

architecture test of Test_FA is
  component FullAdder
    port (a, b, c_in: in std_logic;
          sum, carry: out std_logic);
  end component;
  signal SA, SB, SSum, SCin, SCout: std_logic;
  constant cycle: time := 10 ns ;
  -- configuration
  for U1:FullAdder use entity
    work.FullAdder(behavioral);
begin
  -- Component Instantiation
  U1: FullAdder port map(a=>SA,
    b=>SB, sum=>SSum, c_in=>SCin,
    carry=>SCout);
  Simulation: process
    begin
      ...
    end process;
end test;

```

110

-110

Simulation and Validation

– Test Architecture

- Process generating input stimuli

```

Simulation: process
  begin
    SA <= '0';
    SB <= '0';
    SCin <= '0';
    wait for cycle;

    SA <= '0';
    SB <= '0';
    SCin <= '1';
    wait for cycle;
    ...
    ...
    SA <= '1';
    SB <= '1';
    SCin <= '1';

    wait;
  end process;

```

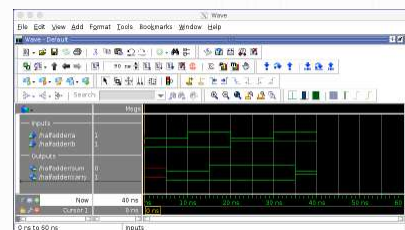
111

-111

Simulation of the FullAdder Testbench

- Write the VHDL testbench
 - sim_FullAdder.vhdl
- Add To Project->Existing File
- Compile the code
- Run the simulation
 - Simulate->Start Simulation
 - Choose work.test_fa.test
 - add wave *
 - run 100

! simulate
this example
on vsim



112

-112

Stimulus Generation

- Stimulus vectors as well as reference vectors for checking
- Stimulus source “on the fly” generation
 - Local constant arrays
 - File I/O
- Clock and reset generation
 - Generally kept separate from stimulus vectors

113

-113

Validation

- Compare reference vectors with response vectors and record errors in external files
- In addition to failed tests record simulation time
- May record additional simulation state

114

-114

The Assert Statement

- Designer can report errors at predefined levels: NOTE, WARNING, ERROR and FAILURE (enumerated type)
- Report argument is a character string written to simulation output

```
assert Q = check(1) and  
        Qbar = check(0)  
report "Test Vector Failed"  
severity error;
```

Example of Simulator Console Output
Selected Top-Level: srbench (behavioral)
: ERROR : Test Vector Failed
: Time: 20 ns, Iteration: 0, Instance: /T1.
: ERROR : Test Vector Failed
: Time: 100 ns, Iteration: 0, Instance: /T1.

- TEXTIO may be faster than ASSERT

```
if yn_int = test_yn then  
    report "Output ok:" & " yn = " & integer'image(yn_int);  
end if;
```

115

-115

Inputs Stored in an Array and Assertions

– Test Architecture

- Input stimuli in an array
- Comparison of results with reference output

```

...
type Stimuli is array (0 to 4) of std_logic;
type ArrayStimuli is array (0 to 7) of Stimuli;

constant FA_table: ArrayStimuli
:= (('0', '0', '0', '0', '0'),
   ('0', '0', '1', '0', '1'),
   ('0', '1', '0', '0', '1'),
   ('0', '1', '1', '0', '0'),
   ('1', '0', '0', '0', '1'),
   ('1', '0', '1', '1', '0'),
   ('1', '1', '0', '1', '0'),
   ('1', '1', '1', '1', '0'));
-- add an error to test assertion

```

Truth Table of the Full Adder

```

...
simulation: process
begin
    wait for cycle;
    for i in FA_table'range(1) loop
        SA <= FA_table(i)(0);
        SB <= FA_table(i)(1);
        SCin <= FA_table(i)(2);
        wait for cycle;
        assert (SCout = FA_table(i)(3))
            report "Problem on carry"
            severity warning;
        assert (SSum = FA_table(i)(4))
            report "Problem on sum"
            severity warning;
    end loop;
    wait;
end process;
...

```

116

-116

Spy Process

- Assertions grouped in a process

```

simulation: process
begin
    wait for cycle;
    for i in FA_table'range(1) loop
        SA <= FA_table(i)(0);
        SB <= FA_table(i)(1);
        SCin <= FA_table(i)(2);
        wait for Cycle;
        Test_SCout <= FA_table(i)(3);
        Test_SSum <= FA_table(i)(4);
    end loop;
    wait;
end process;

spy: Process (SSum , SCout)
begin
    assert (SCout = Test_SCout)
        report "Problem on carry"
        severity warning;
    assert (SSum = Test_SSum)
        report "Problem on sum"
        severity warning;
end process;

```

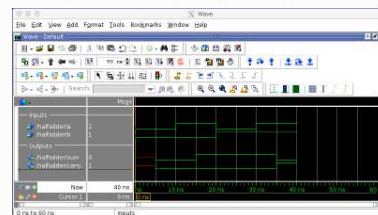
117

-117

Simulation of the FullAdder Testbench

- Write the VHDL **full validation** testbench
 - sim_FullAdder_fullvalidate.vhdl
- Add file to project
- Compile it
- Run the simulation

! simulate
this example
on vsim



```
# ** Warning: Problem on sum
#   Time: 90 ns  Iteration: 0  Instance: /test_fa2
```

118

-118

Inputs Stored in an File

– Test Architecture

- Read input stimuli in a file
- Comparison of results with reference output

```
Simulation: process
  file InputFile: integer is in "input.txt";
  file OutputFile: integer is in "VecteursOUT";
  variable l: line;
  variable VA, VB, VCin, V Cout, V Out: integer;
  variable Tps: integer;
  variable Cycle: time := 10 ns;
begin
  wait for Cycle;
  readline(InputFile, l);
  read(l, Tps);
  Cycle := Tps ns;
```

```
while not endfile(InputFile) loop
  read(l, VA);
  read(l, VB);
  read(l, VCin);
  read(l, V Cout);
  read(l, V Out);
```

```
  SA <= Integer2Bit(VA);
  SB <= Integer2Bit(VB);
  SCIn <= Integer2Bit(VCin);
  wait for Cycle;
  assert (SCout = Integer2Bit(V Cout))
    report "Problem on Cout"
    severity warning;
  assert (SOut = Integer2Bit(V Out))
    report "Problem on S"
    severity warning;
end loop;
wait ;
end process;
```

119

-119

Simulation and Validation

- Synchronous Components
 - Clock process

```
-- alternative  
SClock <= not (SClock) after 50 ns;
```

```
Simulation: process  
begin  
    ...  
    ...  
end process;  
  
Clock: process  
begin  
    SClock <= '0' ;  
    wait for 50 ns;  
    SClock <= '1' ;  
    wait for 50 ns;  
end process;
```

120

-120

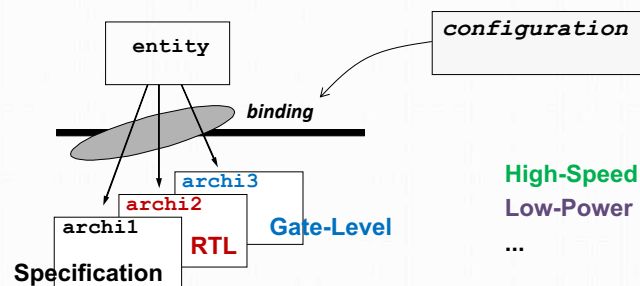
Configurations

121

-121

Configurations

- A design entity can have multiple alternative architectures
- A configuration specifies the architecture that is to be used to implement a design entity



122

-122

Default Binding Rules

- Search for entity with the same component name
- If multiple entities exist, bind the last compiled architecture for that entity
- How do we get more control over binding?

```
architecture structural of serial_adder is
  component comb is
    port (a, b, c_in : in std_logic;
          z, carry : out std_logic);
  end component comb;

  component dff is
    port (clk, reset, d : in std_logic;
          q, qbar : out std_logic);
  end component dff;
  signal s1, s2 : std_logic;

begin
  C1: comb port map (a => a, b => b, c_in => s1, z => z, carry => s2);
  D1: dff port map (clk => clk, reset => reset, d => s2, q => s1, qbar => open);
end architecture structural;
```

123

-123

Configuration Specification

- We can specify any binding where ports and arguments match

```
architecture structural of full_adder is
    --declare components here
    signal s1, s2, s3: std_logic;
    -- configuration specification
    for H1: half_adder use entity WORK.half_adder (behavioral);
    for H2: half_adder use entity WORK.half_adder (structural);
    for O1: or_2 use entity POWER.lpo2 (behavioral)
        generic map(gate_delay => gate_delay)
        port map (I1 => a, I2 => b, Z=>c);
    begin -- component instantiation statements
        H1: half_adder port map (a => In1, b => In2, sum => s1, carry=> s2);
        H2: half_adder port map (a => s1, b => c_in, sum => sum, carry => s2);
        O1: or_2 port map(a => s2, b => s3, c => c_out);
    end structural;
```

library name

entity name

architecture name

124

-124

Configuration Specification

- Short form where applicable
 - for all: half_adder use entity WORK.half_adder (behavioral);
- Delayed binding when a specification is not present
 - Will be available at a later step
 - Analogous to unresolved symbol references during compilation of traditional programs

125

-125

Configuration Declaration

- Written as a separate design unit
- Can be written to span a design hierarchy
- Use of the “for all” clause

```
configuration Config_A of full_adder is -- name the configuration
-- for the entity
for structural -- name of the architecture being configured
    for H1: half_adder use entity WORK.half_adder (behavioral);
    end for;
--
    for H2: half_adder use entity WORK.half_adder (structural);
    end for;
--
    for O1: or_2 use entity POWER.lpo2 (behavioral)
        generic map(gate_delay => gate_delay)
        port map (I1 => a, I2 => b, Z=>c);
    end for;
--
end for;
end Config_A;
```

126

-126

Simulation of the FullAdder Testbench

- Change the architecture to be simulated in
– sim_FullAdder_validate.vhdl

```
for U1:FullAdder use entity
    work.FullAdder(dataflow);
```

 simulate
this example
on vsim

- Add file to project
- Compile it
- Run the simulation

127

-127

Fil Rouge Example: FIR filter

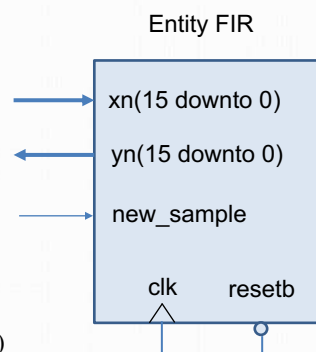
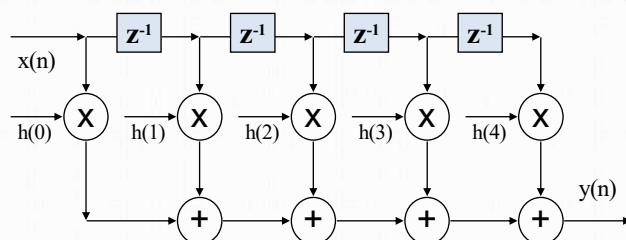
128

-128

Fil Rouge Example: FIR filter

- N-tap Finite Impulse Response

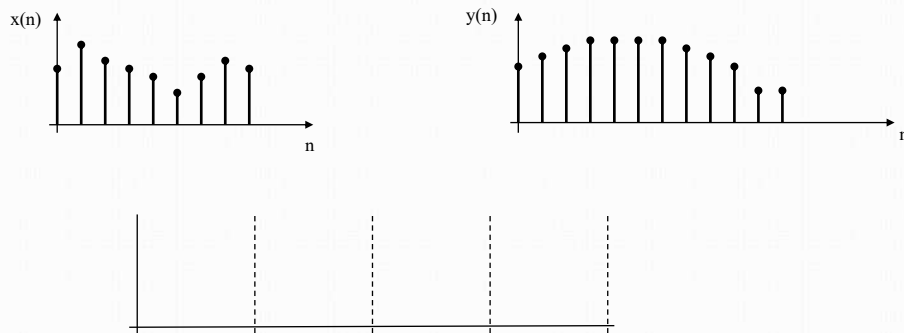
$$y(n) = \sum_{i=0}^{N-1} h(i) \cdot x(n-i)$$



129

-129

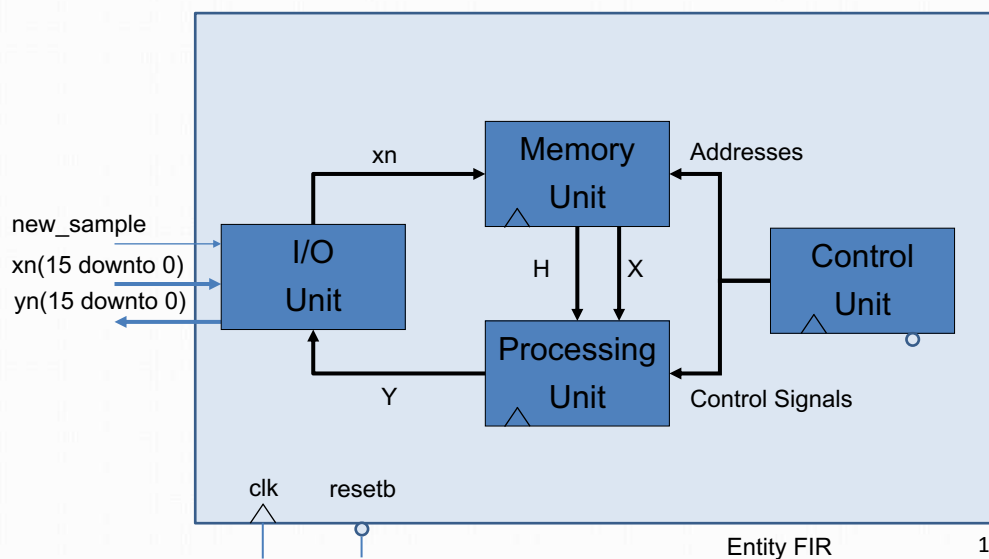
Fil Rouge Example: FIR filter



130

-130

FIR Filter: RTL View



131

-131

FIR Filter: Processing Unit



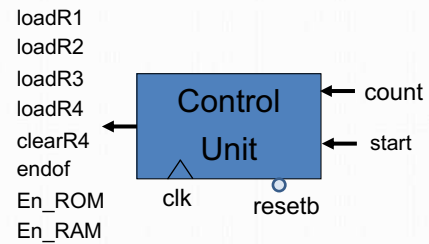
-132

FIR Filter: Memory Unit



–133

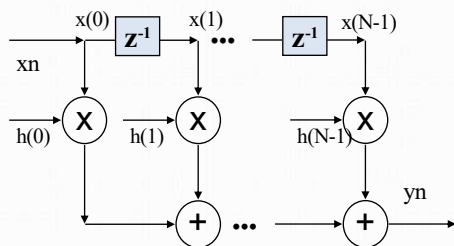
FIR Filter: Control Unit



134

-134

FIR Filter: VHDL Specification



```

ENTITY fir IS
    PORT (xn:IN INTEGER; yn:OUT INTEGER);
END fir;

ARCHITECTURE behavioral OF fir IS
    type MEM is array (0 to N-1) of INTEGER;
    constant sampling_period: time := 1 ns;
BEGIN
    PROCESS
        VARIABLE h,x: MEM;
        VARIABLE tmp: INTEGER;
    BEGIN
        tmp := xn * h(0);
        FOR i IN 1 TO N-1 LOOP
            tmp := tmp + x(i) * h(i);
        END LOOP;
        yn <= tmp;
        FOR i IN N-1 DOWNTOW 2 LOOP
            x(i) := x(i-1);
        END LOOP;
        x(1) := xn;

        WAIT FOR sampling_period;
    END PROCESS;
END behavioral;

```

135

-135

Testbench of FIR Specification

```

package firtypes is
    constant n : integer := 8;
    type mem is array (0 to n-1) of integer;
    constant sampling_period: time := 100 ns;
end firtypes;

library work;
use work.firtypes.all;
entity fir is
    port (xn:in integer; yn:out integer);
end fir;
architecture specification of fir is
begin
    process
        variable x: mem := (others => 0);
        variable h: mem := (13, 0, -38, 51, -38, 0, 13, 0);
        variable tmp: integer;
        begin
            ...
        end process;
    end specification;

```

```

tmp := xn * h(0);
for i in 1 to n-1 loop
    tmp := tmp + x(i) * h(i);
end loop;
yn <= tmp;
for i in n-1 downto 2 loop
    x(i) := x(i-1);
end loop;
x(1) := xn;
wait for sampling_period;

```

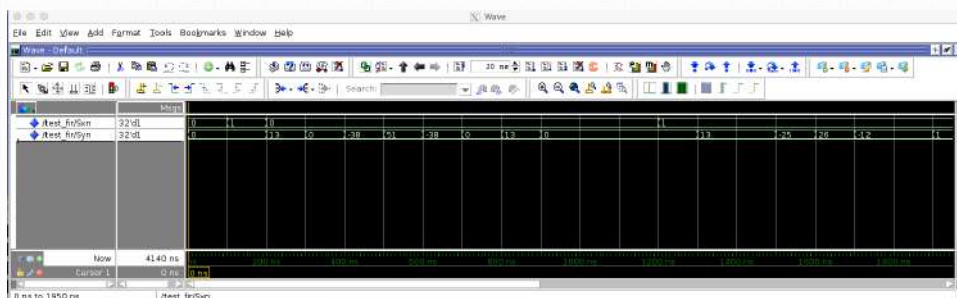
136

-136

Simulation of FIR Specification

- Write the testbench
- Simulate the FIR specification

! simulate this example on vsim



137

-137

Logic Synthesis

138

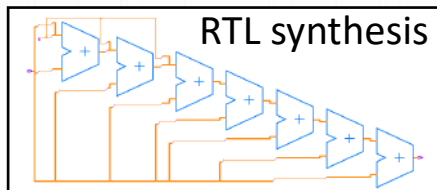
-138

Logic Synthesis at a Glance

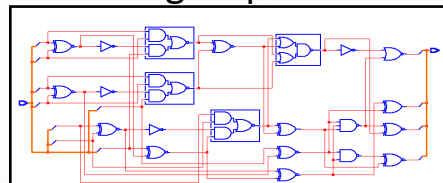
```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_ARITH.all;
entity nb2 is
    port(DIN: in std_logic_vector(7 downto 0);
          nb: out integer range 0 to 8);
end nb2;
architecture arch of nb2 is
begin
    process(DIN)
        variable nb_int: integer range 0 to 8;
    begin
        nb_int := 0;
        for i in 0 to 7 loop
            nb_int := nb_int + conv_integer(DIN(i));
        end loop;
        nb <= nb_int;
    end process;
end arch;
```

VHDL specification

$$nb = \sum_{i=0}^7 DIN(i)$$



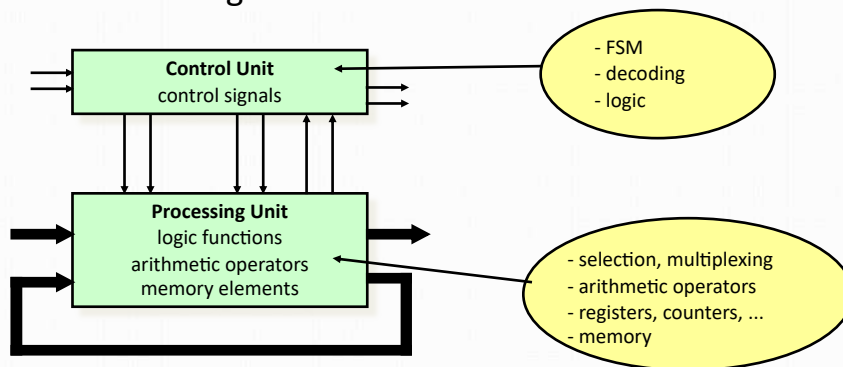
Logic optimization



-139

Logic Synthesis

- Register-Transfer Level (RTL) Specifications
 - Specification of the logic/arithmetic behavior between clocked registers



140

-140

Advantages of Logic Synthesis (over gate-level design)

- Design flow automation
- Higher level of abstraction: more complex designs
- Hardware Description Language (HDL)
- Independent of technology (more or less...)
- **Constrained design flow**
 - timing, power, delay, area
- **Optimized and analyzed results**

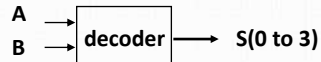
141

-141

Coding Style

entity decoder is

```
port ( A, B : in bit;
      S  : out bit vector (0 to 3) );
end decoder ;
```



```
architecture behavioral
of decoder is
begin
  process (A,B)
  begin
    if A='0' then
      if B='0' then
        S<= "0001"
      else
        S<= "0010"
      end if;
    else
      if B='0' then
        S<= "0100"
      else
        S<= "1000"
      end if;
    end if;
  end process ;
end behavioral;
```

```
architecture dataflow
of decoder is
begin
  S(0) <= not(A) and not(B) ;
  S(1) <= not(A) and B ;
  S(2) <= A and not(B) ;
  S(3) <= A and B ;
end dataflow;
```

```
architecture structural
of decoder is
component DEC24
port ( I1, I2 : in bit;
      O1, O2, O3, O4 : out bit );
end component ;
begin
  CELL : DEC24
  port map (A,B,S(0),S(1),S(2),S(3));
end structural ;
```

142

-142

VHDL RTL coding style for synthesis

```
entity counter is
port (reset: in bit;
      S: out integer := 0);
end counter;

architecture behavioral of counter is
begin
  process
  variable count: integer := 0;
  begin
    wait until reset = '1' for 20 ns;
    if reset = '1' or count = 15
    then count :=0;
    else count := count + 1 after 10 ns;
    end if;
    S <= count;
  end process;
end behavioral;
```

Non Synthesizable

```
entity counter is
port (reset: in bit;
      clk: in bit;
      S: out integer range 0 to 15 );
end counter;

architecture RTL of counter is
signal count: integer range 0 to 15;
begin
  process(reset,clk)
  begin
    if reset='1' then count <=0;
    elsif clk'event and clk='1' then
      if count=15 then count <= 0;
      else count <= count + 1;
      end if;
    end if;
  end process;
  S <= count;
end RTL;
```

Synthesizable

143

-143

VHDL RTL coding style for synthesis

Data types:	Integer (with range) Enumerate, Record, Subtype 1D array of finite dimension Bit, Bit_Vector ou STD_LOGIC, STD_LOGIC_VECTOR Physical, Real, Access, File: ignored
Entity:	<i>in, out, inout</i> , default values are ignored
Packages:	Collection of resources: types, constants, functions, components Standard packages (STD_LOGIC) or technology (design kit)
Declarations:	<i>Constant, Signal, Variable, Component</i> Register, Bus, Linkage, Alias: ignored
Operators:	Logic (and, nand, or, nor, xor, not) Comparison (=, /=, <, >, <=, >=) Arithmetic (+, -, *, sign, abs) (/, mod, rem, ** for a power of 2)
Attributes:	'length, 'event, 'left, 'right, 'high, 'low, 'range, 'reverse_range

144

-144

VHDL RTL coding style for synthesis

Sequential Instructions (process)

Wait:	Supported at the first line of a synchronous PROCESS wait until clock = value; wait until clock'event and clock = value; wait until not clock'stable and clock = value;
--------------	--

Assignment:	Assignment of variables and signals: supported Functions and procedures: supported Transport after: ignored
--------------------	---

If/Case:	Supported
-----------------	-----------

Loops:	<i>for</i> loop with static size: supported <i>while</i> loop: not supported
---------------	---

Parallel Instructions (architecture)

Process:	Sensitivity list or wait for synchronization
Affectations:	Conditional assignment of signals (<i>when, select</i>): supported
Block:	Guarded blocks: not supported
Instantiation:	Port map, generic map, generate: supported

145

-145

IEEE Standard Logic 1164 Package

```

PACKAGE std_logic_1164 IS

  TYPE std_ulogic IS (
    'U', -- Uninitialized
    'X', -- Forcing Unknown
    '0', -- Forcing 0
    '1', -- Forcing 1
    'Z', -- High Impedance
    'W', -- Weak Unknown
    'L', -- Weak 0
    'H', -- Weak 1
    '-' -- Don't care );

  TYPE std_ulogic_vector IS ARRAY ( NATURAL RANGE <> ) OF std_ulogic;

  FUNCTION resolved ( s : std_ulogic_vector ) RETURN std_ulogic;

  SUBTYPE std_logic IS resolved std_ulogic;
  TYPE std_logic_vector IS ARRAY ( NATURAL RANGE <> ) OF std_logic;

```

146

-146

VHDL RTL coding style for synthesis

Conditional and parallel assignment of signals

S <= a+b;	S <= a and b	when a < "010"	with a select
	else a xor b	when a < "101"	S <= a and b
	else a or b;		a xor b
			a or b

when 2 downto 0,
when 3 to 4,
when others;

Combinational Process

```

add: process (a,b,c)
begin
  if c='1' then res <= a + b;
  else res <= a - b;
  end if;
end process;

```

- All **read signals** must be placed in the sensitivity list of the Process
- All **outputs** must be assigned for **all possible values** of the conditions

Synchronous Process

```

process (reset, clock) is begin
  if reset='0' then
    -- actions during asynch. reset
  elsif clock'event and clock='1' then
    -- synchronous statements
    -- no possible else
  end if;
  -- no more possible actions
end process;

```

- Sensitivity list of the Process must contain the **clock** signal and eventually an **asynchronous signal** (reset)
- (Sensitivity list can be replaced by a wait statement)
- Use the template on the left
- Outputs must be assigned both in synchronous and asynchronous manners

-147

And then synthesis tools will do the job for you

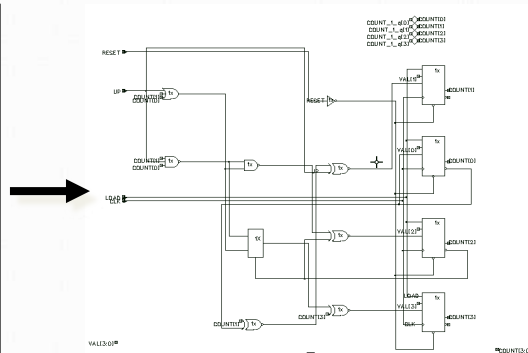
RTL Level Code

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity counter is
    port (reset, clk, load, up: in Std_Logic;
          val: in Std_Logic_Vector(3 downto 0);
          count : buffer Std_Logic_Vector(3 downto 0));
end counter;

architecture RTL of counter is
begin
    synchronous: process(reset,clk)
    begin
        if reset='1' then
            count <= "0000";
        elsif clk'event and clk='1' then
            if load = '1' then
                count <= val;
            elsif up = '1' then
                count <= count + "0001";
            else
                count <= count - "0001";
            end if;
        end if;
    end process;
end RTL;
    
```



Gate-Level Netlist

```

entity counter is
    port (...);
end counter;
architecture SYN_RTL of counter is
    component MUX21LL
        port (A, B, S: in std_logic; Z: out std_logic);
    end component;
    component FD2QLLP
        port (CD, CP, D: in std_logic; Q: out std_logic);
    end component;
end component;
    
```

```

begin
    ...
    U43 : MUX21LL port map( A => tqch, B => data, S => n90, Z => n89);
    U44 : MUX21LL port map( A => tich, B => tqch, S => n90, Z => n88);
    U45 : AN2LL port map( A => rstb, B => load_Fs, Z => n71);
    FF_regx0x : FD2QLLP port map( CD => rstb, CP => clk, D => n89, Q => tqch);
    FF_regx1x : FD2QLLP port map( CD => rstb, CP => clk, D => n88, Q => tich);
    ...
end SYN_RTL;
    
```

148

-148

Logic Synthesis of Combinational Logic

- Logic gates
- Multiplexer, tri-state
- Arithmetic operators
- Decoding

149

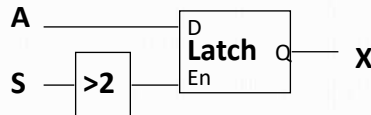
-149

Be **very** Careful with Latches and Logic

Latch

```
process (A,S)
begin
  if S > 2 then
    X <= A;
  end if;
end process;
```

=>

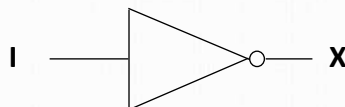


Logic gates

X <= not I

```
process (I)
begin
  if I = '0' then
    X <= '1';
  else
    X <= '0';
  end if;
end process;
```

=>



Complete specification of conditions to provide the full truth table

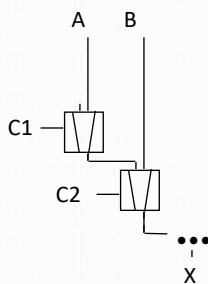
150

-150

Multiplexers

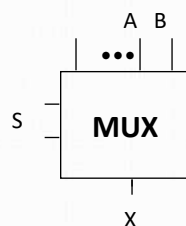
With priority

```
process (A,B,...C1,...)
begin
  if C1 then
    X <= A;
  elsif C2 then
    X <= B;
  ...
  else
    end if;
end process;
```



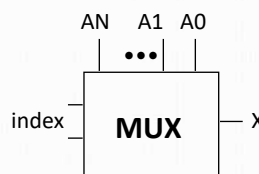
Without priority

```
process (A,B,...,S)
begin
  case (S) is
    when C1 =>
      X <= A;
    when C2 =>
      X <= B;
    ...
  end case;
end process;
```



Array Index

X <= A(index);



151

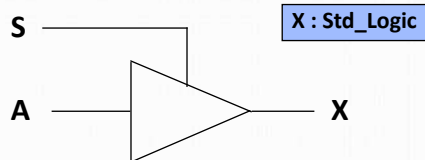
-151

Tri-states and Loops

Tri-State

```
process (A,S)
begin
  if S = '1' then
    X <= A;
  else
    X <= 'Z';
  end if;
end process;
```

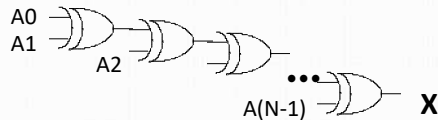
=>



Loop and Generate

```
parité : process (A)
  variable result : bit;
begin
  result := '0';
  for i in 0 to N-1 loop
    result := result xor A(i);
  end loop;
  X <= result;
end process;
```

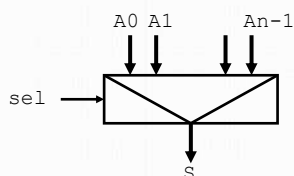
=>



152

-152

Example: data selection

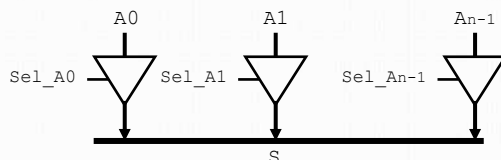


Multiplexer Logic

```
S <= A0 when sel=0 else
  A1 when sel=1 else
  A2 when sel=2 else
  'X';
```

or

```
process(sel, A0,A1,A2)
begin
  case sel is
    when 0 =>
      S <= A0;
    when 1 =>
      S <= A1;
    when 2 =>
      S <= A2;
    when others => S <= 'X';
  end case;
end process;
```



Tri-state Logic

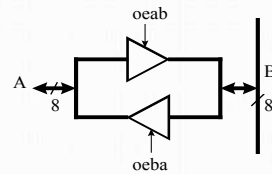
```
signal S, A0, A1,A2 : std_logic ;
signal Sel_A0,Sel_A1,Sel_A2 : std_logic;
...
S <= A0 when sel_A0='1' else 'z';
S <= A1 when sel_A1='1' else 'z';
S <= A2 when sel_A2='1' else 'z';.
```

153

-153

Example: Bidirectional Buffer

- Bidirectional buffer



```
library IEEE;
use IEEE.std_logic_1164.all;
entity transceive is
    port( A,B: inout std_logic_vector(7 downto 0);
          oeab, oeba: in std_logic);
end entity
architecture RTL of transceive
begin
    B <= A when oeab = '1' else "ZZZZZZZZ";
    A <= B when oeba = '1' else (others => 'Z');
end RTL;
```

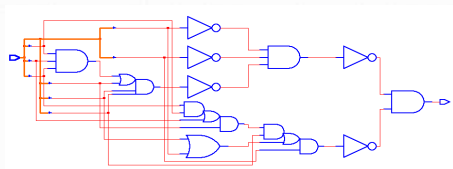
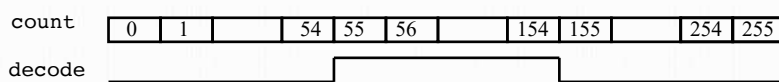
154

-154

Example: Signal Decoding

- Signal decoding

signal count: integer range 0 to 255;



155

-155

Example: Signal Decoding

156

-156

Arithmetic Operators

- Arithmetic operations
+, -, *, and **, / (with restrictions)

$S \leq A \text{ op } B;$

– Signed

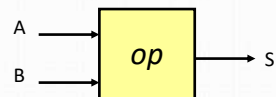
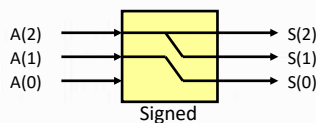
- integer range -128 to 127;
- signed(7 downto 0);

– Unsigned

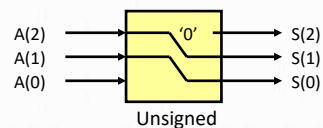
- integer range 0 to 255;
- unsigned(7 downto 0);

- Division: divider is a power of 2

$S \leq A/2;$



```
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_arith.all;
use IEEE.STD_LOGIC_SIGNED.all;
or
use IEEE.STD_LOGIC_UNSIGNED.all;
```

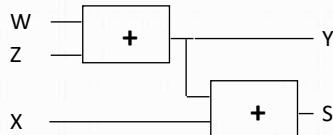


157

-157

Parallel Statements and Assignments

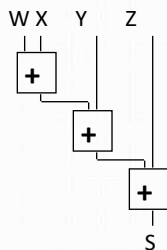
```
architecture A of E is
begin
  S <= X + Y;
  Y <= Z + W;
end A;
```



- Dataflow specifications
- Order of equations has no influence
- Combinational logic

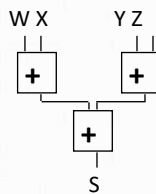
```
S <= W + X + Y + Z;
```

Left to right evaluation



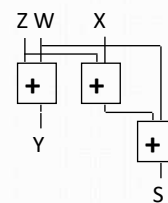
```
S <= (W + X) + (Y + Z);
```

Effect of parentheses



```
Y <= Z + W;
S <= X + Z + W;
```

What you write is what you have



158

-158

Logic Synthesis of Sequential Logic

- Flip-flops, registers
- FSM
- Memory

159

-159

Sequential Logic

- Edge-triggered flip-flop (FF) or registers
 - Sensitivity of the process is on the clock edge
 - value 'mem' could be a vector of bit/integer

```

register: process(clock)
  if (clock'event and clock = '1') then
    reg <= input_val;
  end if;
end process;

```

- Latch
 - Sensitivity of the process is on a signal level

```

latch: process(enable, input_val)
  if enable = '1' then
    mem <= input_val;
  end if;
end process;

```

160

-160

Sequential Logic

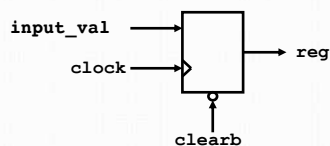
- Asynchronous or synchronous reset (or set)

asynchronous

```

process(clearb, clock)
  if clearb = '0' then
    reg <= 0;
  elsif (clock'event and clock = '1') then
    reg <= input_val;
  end if;
end process;

```

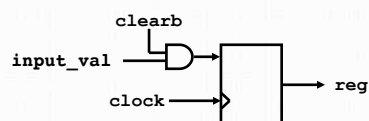


synchronous

```

process(clock)
  if (clock'event and clock = '1') then
    if clearb = '0' then
      reg <= 0;
    else
      reg <= input_val;
    end if;
  end if;
end process;

```



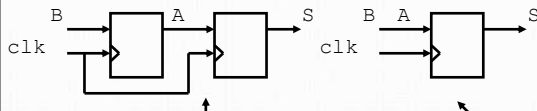
161

-161

Variable or Signals?

Éléments de mémorisation

A signal assigned inside a [clk'event and clk='1'] statement will always be a direct output of a flip-flop



```
signal clk, B, S: std_logic;
.....
begin
...
process(hor1)
  variable A: std_logic;
begin
  if (clk'event and clk='1') then
    S <= A;
    A := B;
  end if;
end process;
```

```
signal clk, A, B, S: std_logic;
.....
begin
...
process(clk)
begin
  if (clk'event and clk='1') then
    A <= B;
    S <= A;
  end if;
end process;
```

```
signal clk, B, S: std_logic;
.....
begin
...
process(clk)
  variable A: std_logic;
begin
  if (clk'event and clk='1') then
    A := B;
    S <= A;
  end if;
end process;
```

For a variable, it depends...

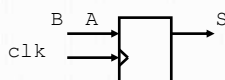
162

-162

Variable or Signals?

Éléments de mémorisation

A signal assigned inside a [clk'event and clk='1'] statement will always be a direct output of a flip-flop



```
signal clk, A, B, S: std_logic;
.....
begin
...
process(clk)
begin
  if (clk'event and clk='1') then
    A <= B;
  end if;
end process;

S <= A;
```

163

-163

Example (Ex3)

Dessiner le schéma logique obtenu par synthèse de la description suivante :

```
process(clk)
  variable int1 : std_logic ;
begin
  if (clk'event and clk='1') then
    int1 := A nand B ;
    int2 <= C nor D ;
    S <= int1 nand int2 ;
  end if ;
end process ;
```

164

-164

Some more rules

Only one clock and
only one edge

```
process(clk)
  if (clk'event and clk = '1') then
    reg <= val1;
  end if;
  if (clk'event and clk = '0') then
    reg <= val2;
  end if;
end process;
```

Do not touch
the clock!

```
process(clk)
  if (clk'event and clk = '1' and ena = '1') then
    reg <= val;
  end if;
end process;
```

Synchronous
register with load

```
process(clk)
  if (clk'event and clk = '1') then
    if (ena = '1') then
      reg <= val;
    end if;
  end if;
end process;
```

165

-165

Counting and Shifting

- Signal declared as
 - integer range 0 to N-1
 - signed/unsigned(n downto 0)
- Binary counter from 0 to N-1

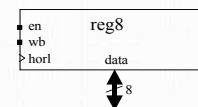

```
if (compteur = N-1) then compteur <= 0;
else count <= count + 1;
end if;
```
- N-bit shift register
 - regdec(N-1 downto 1) <= regdec(N-2 downto 0);
 - regdec(0) <= '0';
 - or
 - regdec <= regdec*2;

166

-166

Example: 8-bit Register

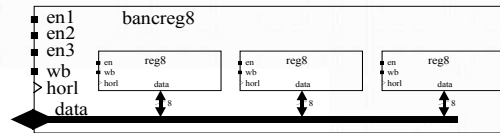
- 8-bit register with bidirectional I/O and high-impedance output
- en = '1': read/write; en = '0': data = 'Z'
- en = '1' AND wb = '0': write
- en = '1' AND wb = '1': read
- data: 8-bit bidirectional I/O



167

-167

Example: Register Bank



168

-168

Memory

- A memory is a 1D-array of words

- Type declaration

```
type t_mem is array(0 to N-1) of std_logic_vector(nb_bits-1 downto 0);
type t_mem is array(0 to N-1) of integer range 0 to 2**nb_bits-1;
```

- Signal declaration

```
signal mem: t_mem;
const rom: t_mem;
```

– mem(i) is the i^{th} element of the memory

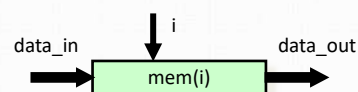
- Read the memory

```
data_out <= mem(i);
data_out_rom <= rom(i);
```

- Write in the memory

– edge or level triggered

```
if clk'event and clk='1 then
    mem(i) <= data_in;
end if;
```



169

-169

Finite State Machines

- Enumeration of states

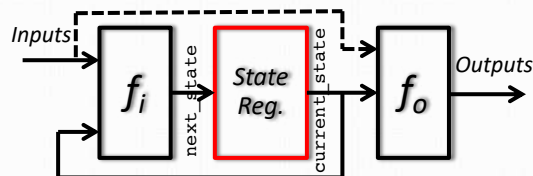
```
type states is (init_state, state1, state2, state3, ...);
```

- State register declaration

```
signal current_state, next_state: states;
```

- State register

– synchronous process



```
StateRegister: process(reset, clk)
begin
    if reset='0' then
        current_state <= init_state;
    elsif (clk'event and clk='1') then
        current_state <= next_state;
    end if;
end process;
```

170

-170

Finite State Machines

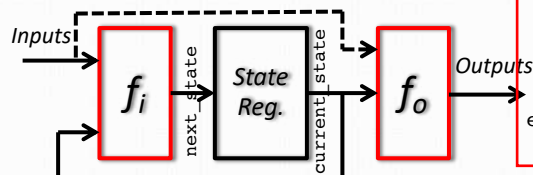
- Decoding logic

– combinational logic

- remember to **respect RTL coding rules**

– Moore/Mealy style

- for Mealy outputs assignments are inside the condition on inputs



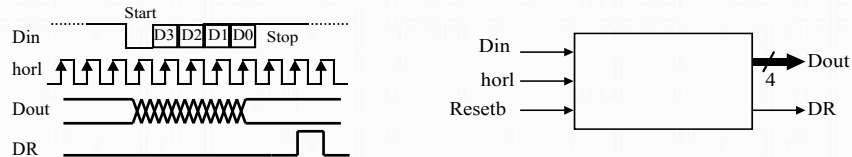
```
moore:process(inputs,current_state)
begin
    case current_state is
        when state1 =>
            outputs <= ...;
            if input1 = ...
            then next_state <= state2;
            else next_state <= state1;
            end if;
        when state2 =>
            outputs <= ...;
            if input2 = ...
            then next_state <= state5;
            else next_state <= state2;
            end if;
        ...
        when others =>
            outputs <= ...;
            next_state <= ...;
    end case;
end process;
```

171

-171

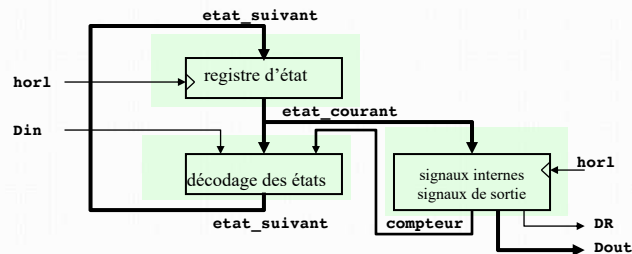
Exercice

Décrire la fonction “ récepteur série asynchrone ” de mots binaires de 4 bits



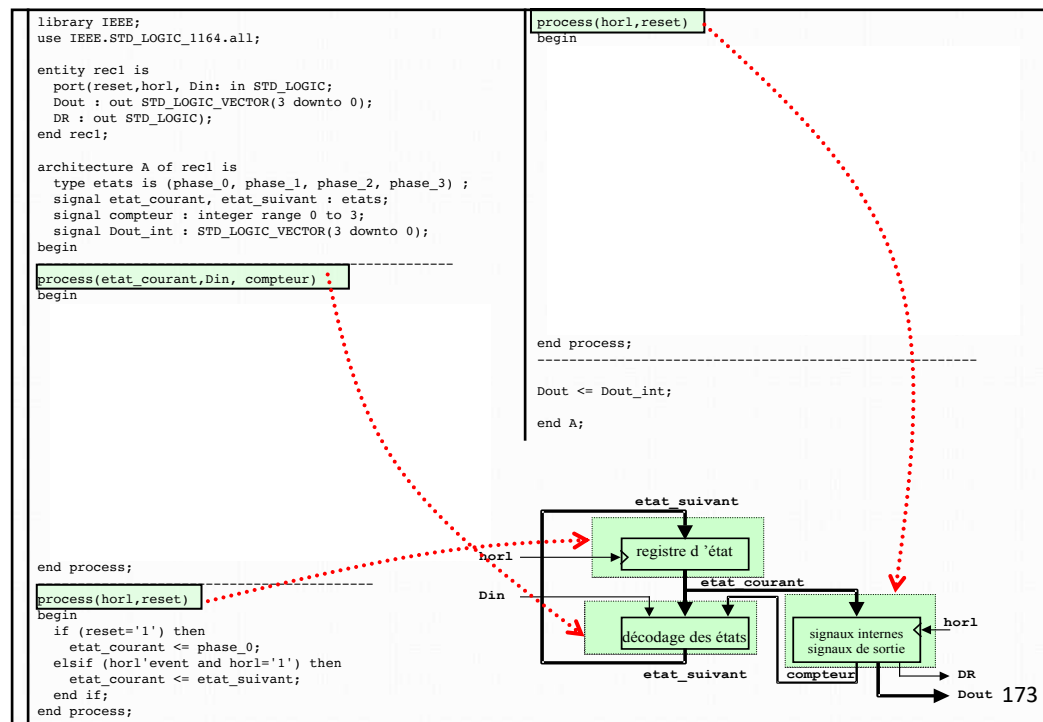
Le message binaire débute par un start bit (Din='0') suivi des 4 bits d'information et se termine par 2 stop bits (Din='1').
Le circuit reconstitue le mot de 4 bits et le présentera après réception sur le bus de sortie Dout en activant le signal DR (Data Ready).

Définir les différentes phases du traitement, attente du start bit, réception des 4 bits,
et décrire sous forme de machine d'états le contrôleur.



172

-172



-173

RT and Logic Synthesis CAD Algorithms

Structuring, Flattening, Mapping

174

-174

Logic Synthesis Principle

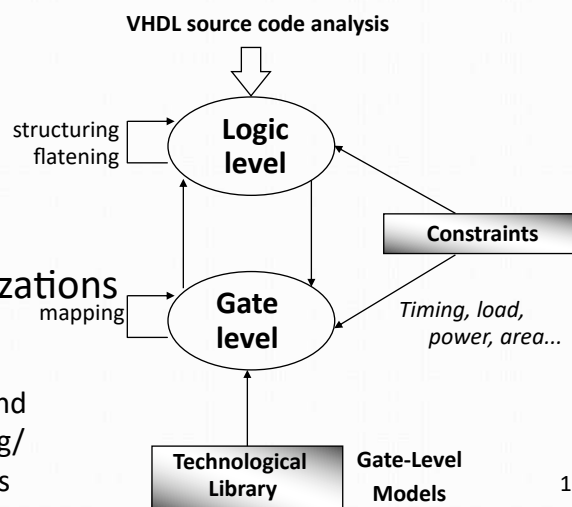
- Logic Synthesis = Translation + Optimizations

- Translation

- VHDL source code is analyzed and transformed into a generic logic representation

- Constrained optimizations

- Logic is mapped onto gates from the technological library and optimized under timing/power/area constraints



175

-175

Optimizations

- Structuring: area oriented
 - Simplify and factorize Boolean equations
- Flattening: speed oriented
 - Distribute factors in Boolean equations
- Mapping
 - Selection of gates in the technological library
 - Specification of constraints (power/delay/load/area)
 - Generate several solutions

176

-176

Optimizations

- Structuring: area oriented
 - Simplify and factorize Boolean equations
 - Share common factors

14 AND
2 NOT
4 OR

$x = a.b.c + a.b.c + a.d.e$
 $y = a.b.c.d + a.b.c.d$
 $z = a.b + c.d$



Simplify

7 AND
2 OR

$x = a.b + a.d.e$
 $y = a.b.c$
 $z = a.b + c.d$

Structuring

But delay is increased!

$t = a.b$
 $u = c.d$
 $x = t + a.d.e$
 $y = t.c$
 $z = t + u$

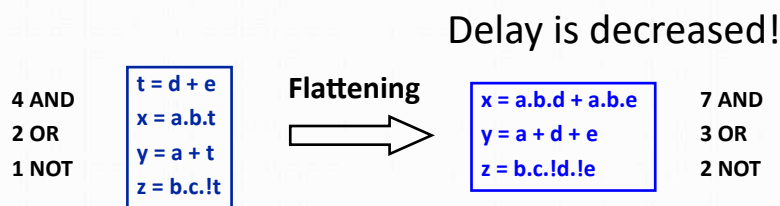
5 AND
2 OR

177

-177

Optimizations

- Flattening: speed oriented
 - Distribute factors in Boolean equations
 - Reduces delay



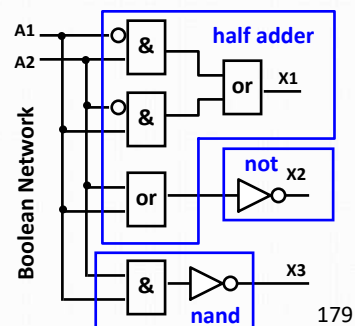
178

-178

Optimizations

- Mapping
 - Selection of gates in the technological library
 - Specification of constraints (power/delay/load/area)
 - Generate several solutions by graph covering
 - Evaluate delay/cost
 - Solution with lowest area respecting delay constraint is retained

Technological Library of Logic Gates (many gates in a' design kit' library)			
gate	delay	cost	function
nand	2 ns	3	$!(AB)$
half adder	4 ns	14	$!A.B + A.!B$
....			



179

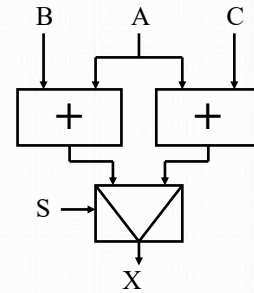
-179

Resource Sharing

```

process (A,B,C,S)
begin
    if (S = '1') then
        X <= A + B ;
    else
        X <= A + C ;
    end if;
end process;

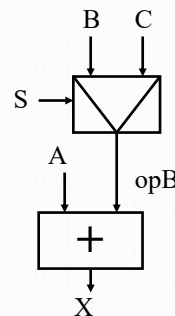
```



```

process (A,B,C,S)
    variable opB : integer;
begin
    if (S = '1') then
        opB := B ;
    else
        opB := C ;
    end if;
    X <= A + opB;
end process;

```



180

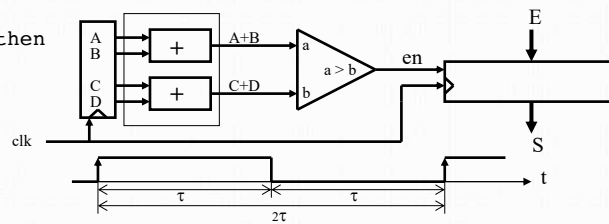
-180

Pipeline

```

if (clk'event and clk = '1') then
    if (A + B) > (C + D) then
        S <= E;
    end if;
end if;

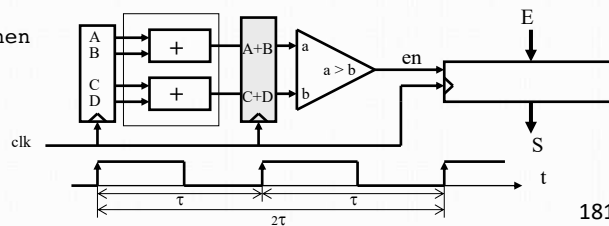
```



```

signal AB, CD: integer;
...
if (clk'event and clk = '1') then
    AB <= A + B;
    CD <= C + D;
    if AB > CD then
        S <= E;
    end if;
end if;

```



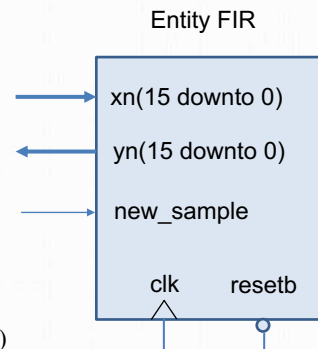
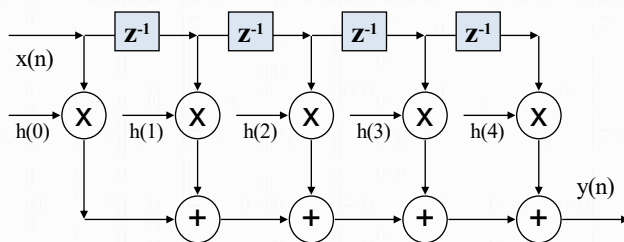
181

-181

Fil Rouge Example: FIR filter

- N-tap Finite Impulse Response

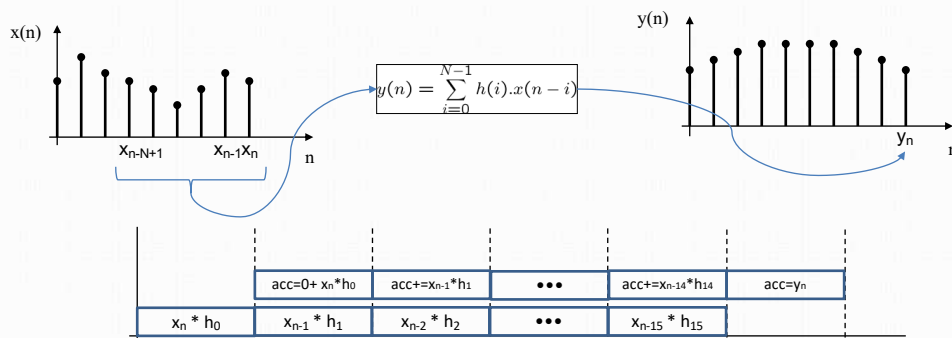
$$y(n) = \sum_{i=0}^{N-1} h(i) \cdot x(n-i)$$



182

-182

FIR filter: Scheduling of Operations



183

-183

FIR Filter: Processing Unit

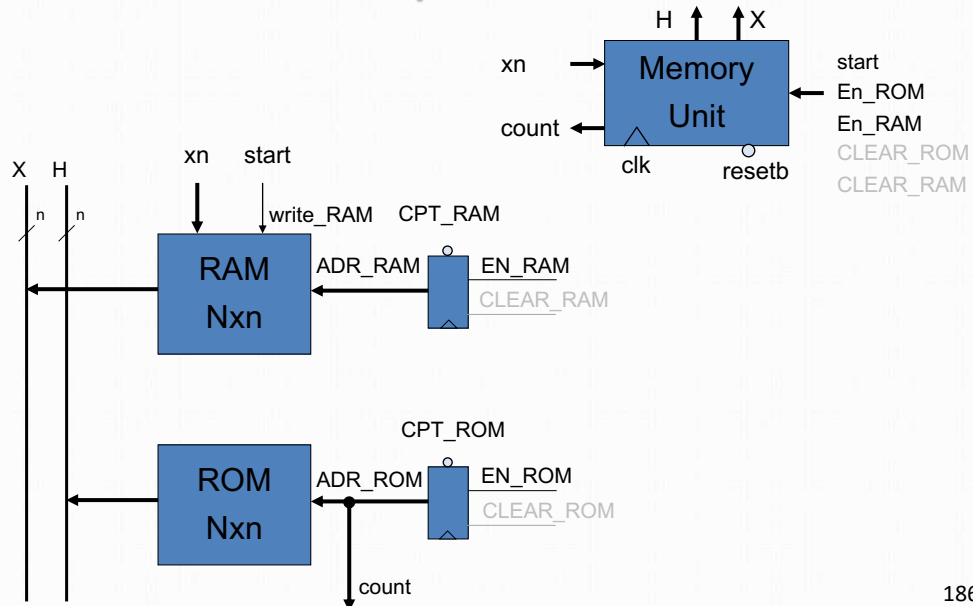
The diagram illustrates the internal structure of an FIR Filter Processing Unit. It features a central Processing Unit, a multiplier (X), and an adder (+). The Processing Unit has inputs H and X, and output Y. It is clocked by clk. The multiplier (X) takes inputs R1 and R2, and outputs M. The adder (+) takes inputs M and R4, and outputs yn. The unit also includes registers R1, R2, R3, and R4. The inputs X and H are n-bit, and the output yn is 2n-bit. The unit is controlled by loadR1, loadR2, loadR3, loadR4, and clearR4 signals.

–184

[illegible]

-185

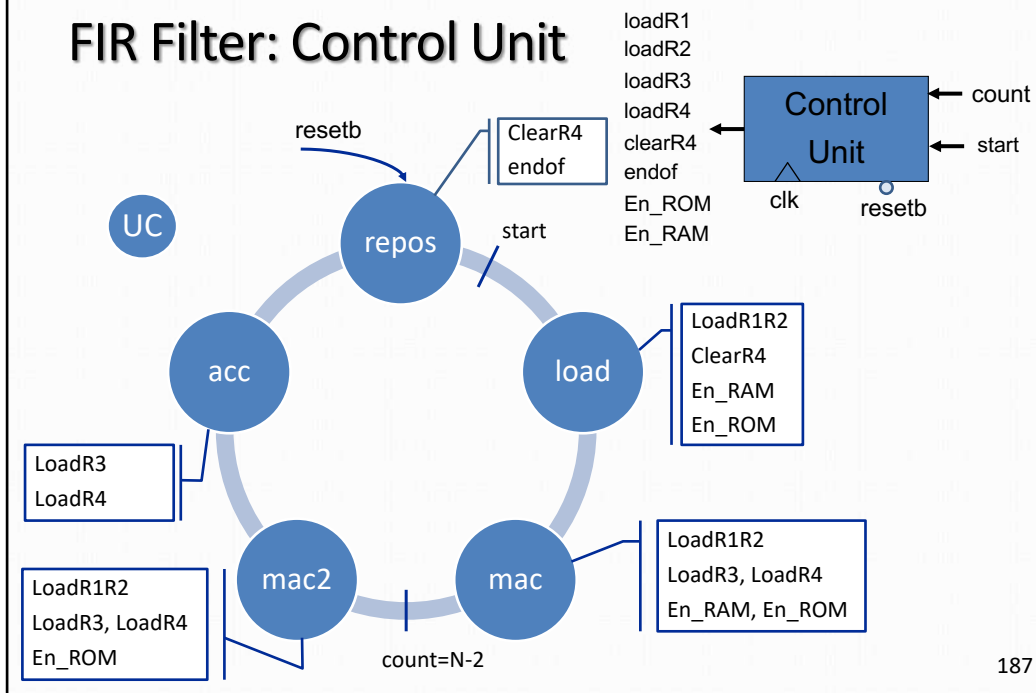
FIR Filter: Memory Unit



186

-186

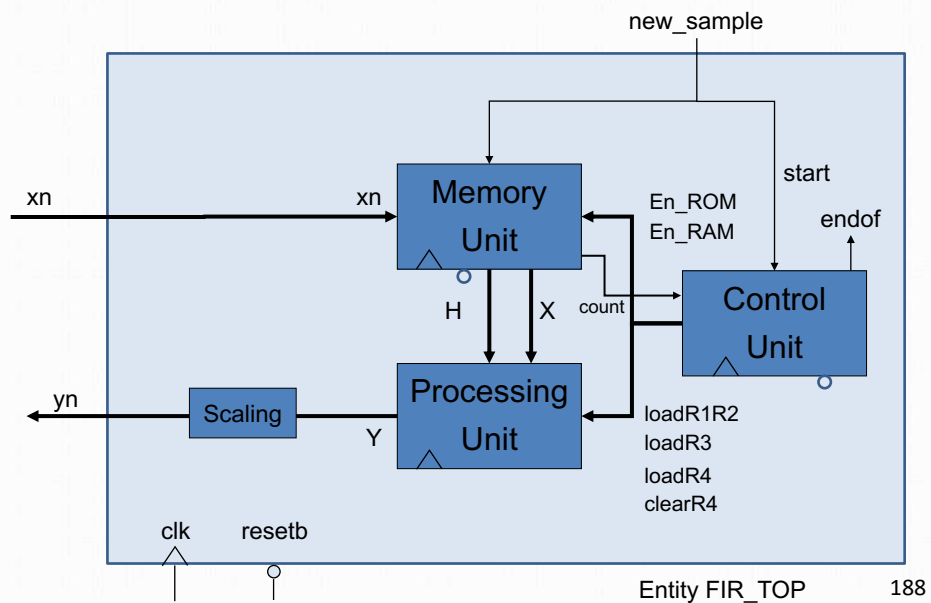
FIR Filter: Control Unit



187

-187

FIR Filter: Top Level



-188

Simulation of FIR RTL: sim_fir_top

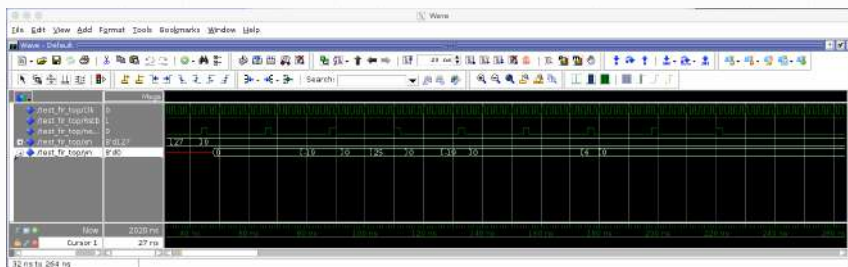
- Write the testbench
 - Stimuli and expected output

! simulate this example on vsim

-- impulse response and then step response

```
xn: (0,127,0,0, 0, 0, 0,0, 0,0,0,0,127,127,127,127,127,127,127,127);
yn: (0,12, 0,-38,50,-38,0,12,0,0,0,0,12, 12, -25,25, -12,-12,0, 0, 0);
```

- Simulate fir_top



189

-189